影像處理 Image Processing

大學智慧機器人學程 | 高中 AI 機器人課程 | 台灣自編大學教科書

編者：李世淵(機器人叫獸)|助教：李天宇、李宇晴

第 15 章 深度學習

本章學習目標

- 說明從手工特徵到學習特徵的典範轉移，理解深度學習「端到端」的核心精神。
- 推導神經網路的基本構件：神經元、激勵函數、損失函數與反向傳播。
- 闡述卷積神經網路(CNN)的三大構件：卷積層、池化層、全連接層，理解其為何適合影像。
- 認識經典 CNN 架構譜系(LeNet→AlexNet→VGG→GoogLeNet→ResNet→MobileNet)的演進脈絡。
- 以三大框架(TensorFlow、Keras、PyTorch)實作 MNIST 手寫數字分類。
- 運用遷移學習與資料增強，在小資料上訓練出高效能的影像模型。

叫獸開講|黃仁勳與 AlexNet：兩顆遊戲顯卡掀起的革命

2012 年 9 月，電腦視覺界發生了一場地震。在權威的 ImageNet 影像辨識競賽(ILSVRC)上，多年來各隊都靠著人工設計特徵配上傳統分類器，辛苦地把錯誤率一個百分點、一個百分點地往下磨，當時的最佳成績卡在約 26% 的 top-5 錯誤率。然後，來自多倫多大學的一支隊伍——Alex Krizhevsky、Ilya Sutskever，以及他們的指導教授、後來被稱為「深度學習教父」的 Geoffrey Hinton——投下了一顆震撼彈：他們的模型把錯誤率一舉壓到 15.3%，足足領先第二名近 11 個百分點。在一個習慣以零點幾個百分點論輸贏的領域，這個差距簡直是不同次元的存在。這個模型，就是後來以第一作者命名的 AlexNet。

AlexNet 憑什麼?它其實是一個八層的卷積神經網路(五層卷積、三層全連接)，

概念上並非全新——它的祖先正是第 14 章 LeCun 讀支票的 LeNet。真正讓它脫胎換骨的是三個關鍵：第一，它用了 ReLU 激勵函數，讓深層網路的訓練快了好幾倍;第二，它用了 Dropout 這個新招來對抗過度擬合(第 14 章的老朋友);第三——也是最出人意料的——它是在兩張 NVIDIA GTX 580 上訓練的。沒錯，是遊戲玩家用來跑 3A 大作的那種顯示卡。

這第三點，正是這個故事最迷人的轉折。原本為了「讓遊戲畫面更逼真」而設計的圖形處理器(GPU)，因為天生擅長大量平行的數值運算，竟意外地與神經網路的訓練需求完美契合——訓練神經網路，本質上就是海量的矩陣乘法，而這正是 GPU 的看家本領。AlexNet 團隊用兩張遊戲顯卡，把原本需要數週的訓練壓縮到五、六天，讓「訓練一個夠深的網路」從不切實際變成了可行。這個發現讓一家原本以電玩顯卡聞名的公司——NVIDIA，以及它的創辦人黃仁勳——站上了 AI 時代的浪頭。黃仁勳多次談到，AlexNet 是 NVIDIA 全力轉向 AI 運算的關鍵時刻：公司從此不再只是賣遊戲顯卡，而是打造整個 AI 運算的基礎設施。今天訓練大型語言模型的資料中心裡，滿滿都是 NVIDIA 的晶片——這條路的起點，就是 2012 年那兩張跑 AlexNet 的 GTX 580。

AlexNet 之後，深度學習如野火燎原：Google、Facebook、微軟、百度紛紛重金投入，ImageNet 的冠軍模型愈疊愈深，到 2015 年 ResNet 的錯誤率已低於人類水準。這場革命的三大燃料——大數據(第 14 章的 ImageNet)、強大算力(GPU)、以及深層網路架構——缺一不可。而它給我們的最大啟示，呼應了第 13、14 章一路鋪陳的主軸：深度學習最革命性的地方，不是它「更準」，而是它讓機器自己學會了特徵。AlexNet 較低的網路層自動長出了邊緣與顏色的偵測器，較高的層則認得出輪子、狗臉——這些過去要靠人類專家嘔心瀝血設計(SIFT、HOG)的東西，網路從資料中自己學會了。這一章，我們就來理解這場革命的技術核心。

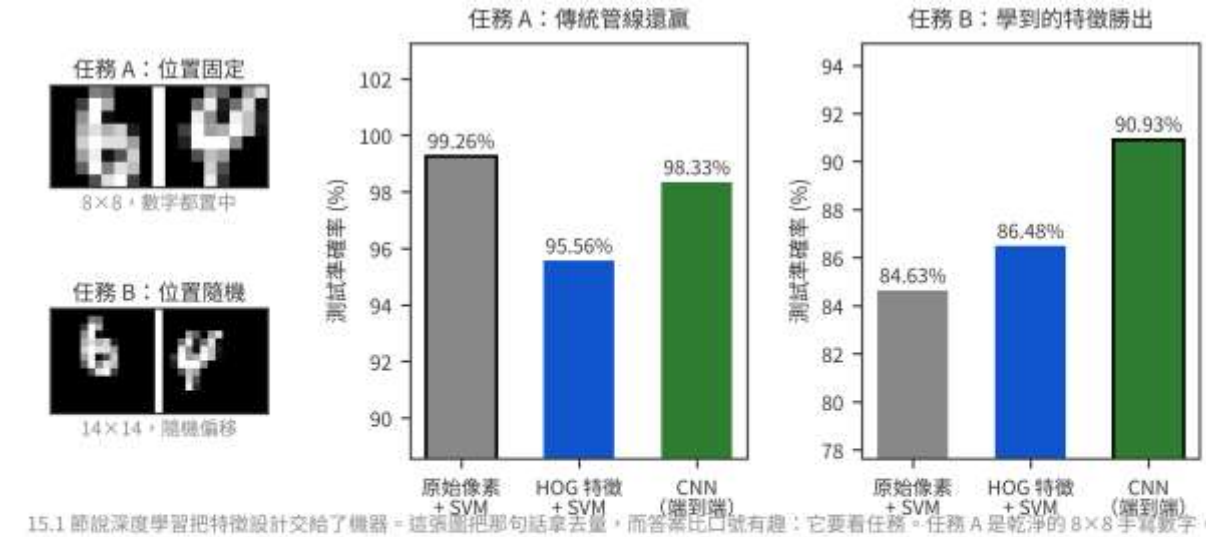
思考題：GPU 原本是為了電玩遊戲的畫面而設計，卻意外成為 AI 革命的引擎。這種「一項技術在原本用途之外找到革命性新應用」的現象，你還能想到哪些例

子?這對「基礎技術的投資」有什麼啟示?

15.1 從手工特徵到學習特徵：典範轉移

第 13.8 節與第 14 章已經多次預告這場典範轉移，現在正式展開。回顧傳統的影像辨識流程(第 14.2 節的五環節)：人類專家設計特徵(第 13 章的 SIFT、HOG)，再把特徵餵給分類器(第 14 章的 SVM)。這裡的瓶頸顯而易見——特徵是人設計的，而人的設計能力有限：我們能想出邊緣、角點、梯度方向，但面對「什麼特徵能區分狗和狼」這種問題，人類的直覺很快就見底了。

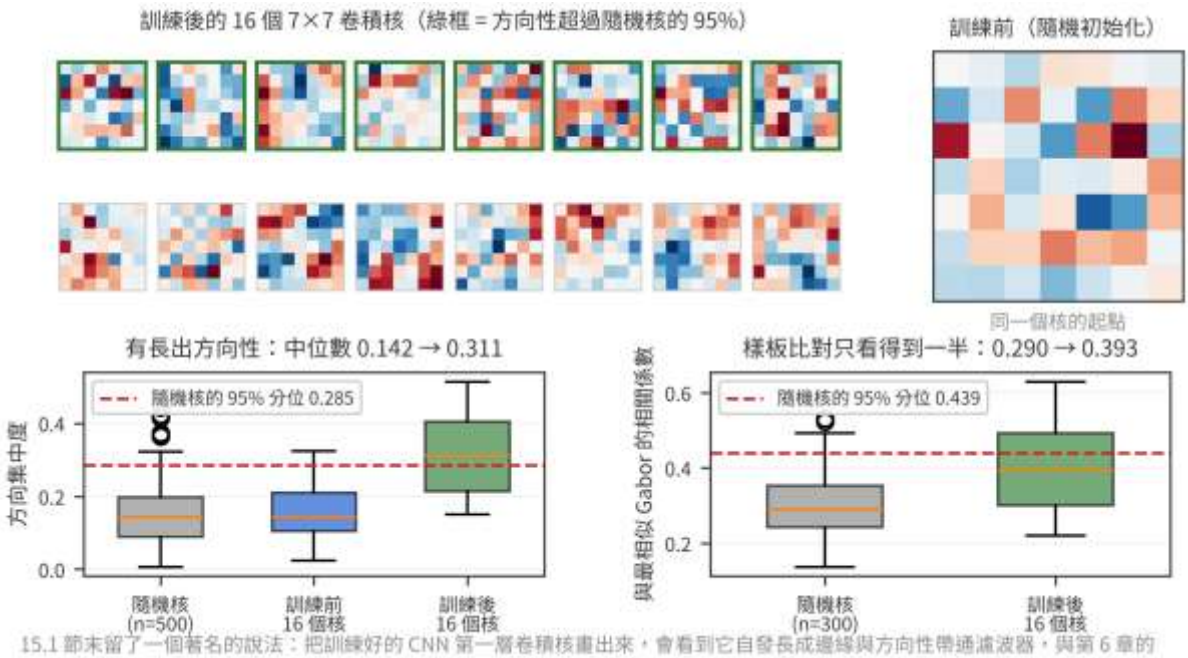
深度學習的核心突破，是把「特徵設計」這件事，從人類手中交給了機器。一個深度神經網路，把「從像素到特徵」與「從特徵到類別」整合進同一個模型，端到端(end-to-end)一起訓練：你只需給它影像與答案，它會自己學出「什麼特徵最有助於分類」。這就是叫獸開講中最關鍵的啟示——AlexNet 的網路層自動長出了邊緣偵測器與物件辨識器，無需人類插手。



15.1 節說深度學習把特徵設計交給了機器。這張圖把那句話拿去量，而答案比口號有趣：它要看任務。任務 A 是乾淨的 8×8 手寫數字，每個數字都好好置中。在這裡，第 13 章的手工特徵配 SVM 拿到 95.56%、連最樸素的「原始像素 + SVM」都有 99.26%，而一個 CNN 端到端學完只有 98.33%——傳統管線還贏。任務 B 只改一件事：把同樣的數字擺到 14×14 畫布的隨機位置上，模擬真實世界裡物件不會乖乖置中。原始像素 + SVM 立刻掉到 84.63%，HOG 也只有 86.48%（已用最佳參數參賽，不是稻草人），而同一個 CNN（僅 6,218 個參數）拿到 90.93%。差別來自 15.3.1 節的權重共享：同一組卷積核滑遍整張影像，數字出現在哪裡都認得;而攤平成向量的做法，位置一變，每個像素的意義就全變了。所以這一章要先立一個誠實的前提：深度學習不是自動比較好，它贏在「任務夠難、變異夠多、資料夠多」的時候。在小而乾淨的問題上，第 13、14 章的工具往往又快又準又好解釋——會選工具，比會用最新的工具更重要。

圖 15-1　典範轉移：什麼時候「學到的特徵」才真的贏

而這正呼應了第 6.6.7 節與第 13.8 節埋下的伏筆：當我們把訓練好的 CNN 第一層卷積核視覺化，會發現它們自發演化成了邊緣與方向性帶通濾波器，與人工設計的 Gabor、SIFT 驚人地相似。這個說法本書做了實測，結果要分兩半講，因為它常被引用得太滿。圖 15-2 用一個刻意不循環的任務(判斷小圖來自哪一張自然照片，完全沒提到邊緣或方向)訓練 CNN，再用與樣板無關的方式量第一層卷積核的「方向集中度」：同一批核訓練前的中位數 0.142(與隨機核無異)，訓練後升到 0.311，16 個核中有 8 個超過隨機核的 95% 分位(純靠運氣的期望值是 0.8 個)——方向性結構確實是學出來的。但若改用「與 Gabor 樣板的相關係數」來量，訓練後中位數 0.393、隨機核 0.290，只有 6 個核超過隨機核的 95% 分位，差距小得多。換句話說：方向性會自發出現，但「長得像教科書上那張 AlexNet 卷積核圖」是另一回事，那需要 ImageNet 等級的自然影像與資料量，不是隨便一個小 CNN 都能重現。帶著這個分寸再讀下去：深度學習沒有推翻前面十四章的智慧，而是「自動重新發明」了它的一部分，並能組合出人類難以手工設計的、更抽象的高層特徵。這也是為什麼要先學完前面各章：唯有理解手工特徵，才能真正看懂 CNN 到底學到了什麼、為什麼有效。



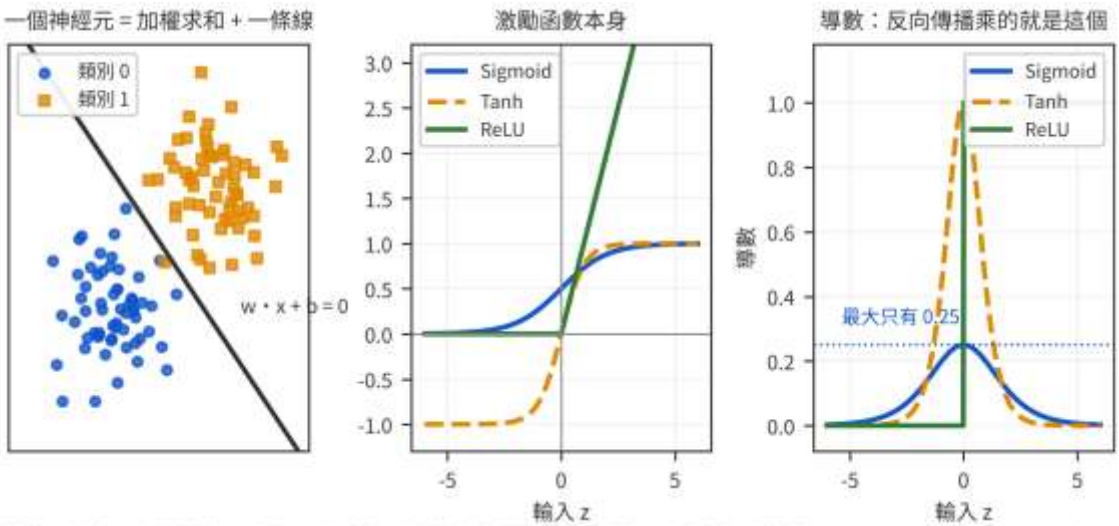
15.1 節末留了一個著名的說法：把訓練好的 CNN 第一層卷積核畫出來，會看到它自發長成邊緣與方向性帶通濾波器，與第 6 章的 Gabor、第 13 章的手工特徵驚人地相似。這張圖把那句話拿去驗，而結果需要分兩半說。實驗設計得刻意不循環：任務是「判斷這塊 24×24 的小圖來自六張自然照片中的哪一張」，完全沒有提到邊緣或方向，影像還先減掉了各自的平均亮度，免得網路靠亮度作弊（測試準確率 88.7%）。好消息在左下：用與樣板無關的方式量「方向集中度」（把核的功率頻譜當角度分布，看它有多集中），同一批核訓練前中位數 0.142（與隨機核的 0.142 無異），訓練後升到 0.311，8/16 個核超過隨機核的 95% 分位（純靠運氣的期望值是 0.8 個）。方向性確實是學出來的，不是初始化就有的。右下則要保守一點：改用「與 Gabor 樣板的相關係數」來量，訓練後中位數從隨機核的 0.290 升到 0.393，只有 6/16 個核超過隨機核的 95% 分位。同一件事，用這個指標看起來弱得多。兩個結論都要講。其一，方向性結構確實會自發出現，這一點與 15.1 節的說法一致，也與第 6 章的 Gabor、第 13 章的手工特徵遙相呼應;其二，「長得像教科書上那張 AlexNet 卷積核圖」是另一回事，那需要 ImageNet 等級的自然影像與資料量，不是隨便一個小 CNN 都能重現——本圖的核有方向，卻選稱不上 Gabor。順帶一提，兩個指標的落差本身就是第 13 章那一課的回馬槍：樣板比對只能看見「長得像我這個樣板」的核，對其他形式的方向性結構是半盲的;換一個不綁樣板的指標，同一個效應就清楚得多。量什麼，往往決定了你會看見什麼。

圖 15-2　第一層真的長出方向性濾波器了嗎

15.2 神經網路的基本構件

15.2.1 神經元與激勵函數

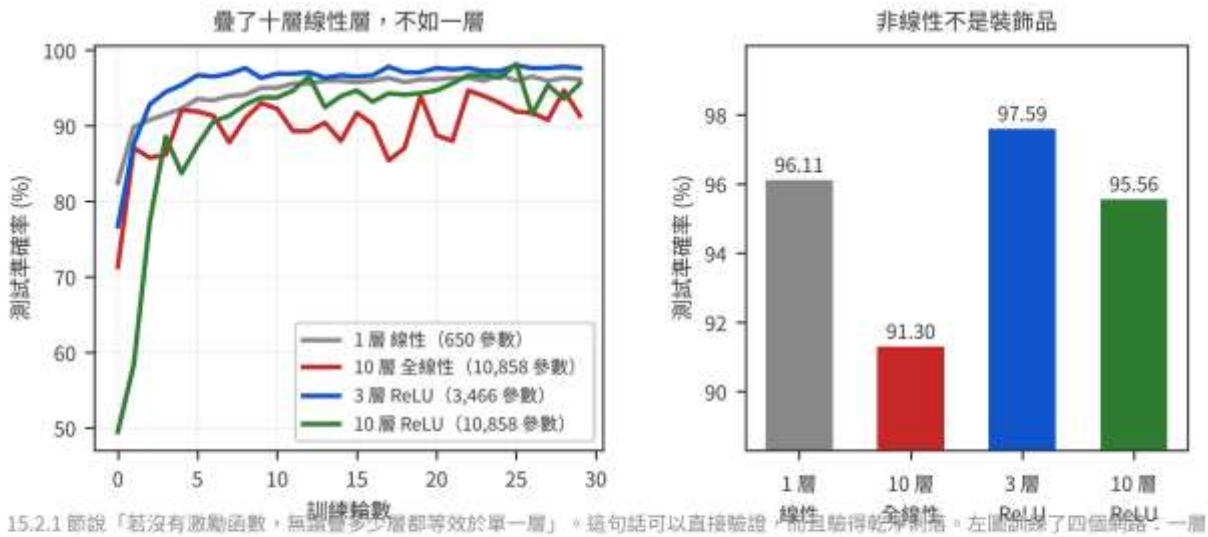
神經網路的靈感來自生物神經元。一個人工神經元做的事很簡單：把多個輸入各自乘上一個權重(weight)、加總、再加上一個偏置(bias)，然後通過一個激勵函數(activation function)輸出。權重與偏置就是神經元「學習」的對象——訓練的過程，就是不斷調整這些數值，讓網路的輸出愈來愈接近正確答案。



15.2.1 節的三句話，一張圖看完。左圖：一個神經元做的事就是把輸入加權求和再加偏置，也就是 $w \cdot x + b$——而 $w \cdot x + b = 0$ 在平面上正是一條直線，在高維空間是一個超平面。這是第 14.4.3 節推導 SVM 時已經見過的同一個式子，差別只在怎麼決定 w 與 b：SVM 解一道有唯一解的最佳化問題，神經網路則靠梯度下降一步步挪。所以單一個神經元的能力只等於一條線性分界，要處理彎曲的邊界，就得靠中間的激勵函數注入非線性，再把許多神經元疊起來。右圖是本章最該記住的一張小圖：三種激勵函數的導數。反向傳播每往回走一層，梯度就要乘上當層激勵函數的導數一次。Sigmoid 的導數最大只有 0.25，而且輸入稍大或稍小就迅速趨近 0;Tanh 最大是 1.0 但兩側一樣會飽和;ReLU 在正半邊的導數固定是 1.0，乘幾層都不會縮小。把 0.25 連乘十層是 0.25 的十次方，大約一兆分之一——這就是「梯度消失」四個字的全部算術，也是 ReLU 取代 Sigmoid 的根本原因。圖 15-9 會把這件事實際量出來。

圖 15-3　一個神經元，與三種激勵函數

激勵函數為網路注入了「非線性」，這一點至關重要：若沒有它，無論疊多少層，整個網路都只是一連串線性運算的組合，等效於單一層，無法學習複雜的規律。常見的激勵函數有：Sigmoid(把輸出壓到 0 與 1 之間，早期常用，但在深層網路會導致梯度消失)、Tanh(類似 Sigmoid 但範圍是 -1 到 1)、以及 ReLU(修正線性單元，負值歸零、正值保留)。ReLU 正是叫獸開講中 AlexNet 的關鍵創新之一——它計算極簡、且大幅緩解了梯度消失問題，讓深層網路的訓練從「幾乎不可能」變得「又快又穩」，至今仍是最主流的選擇。

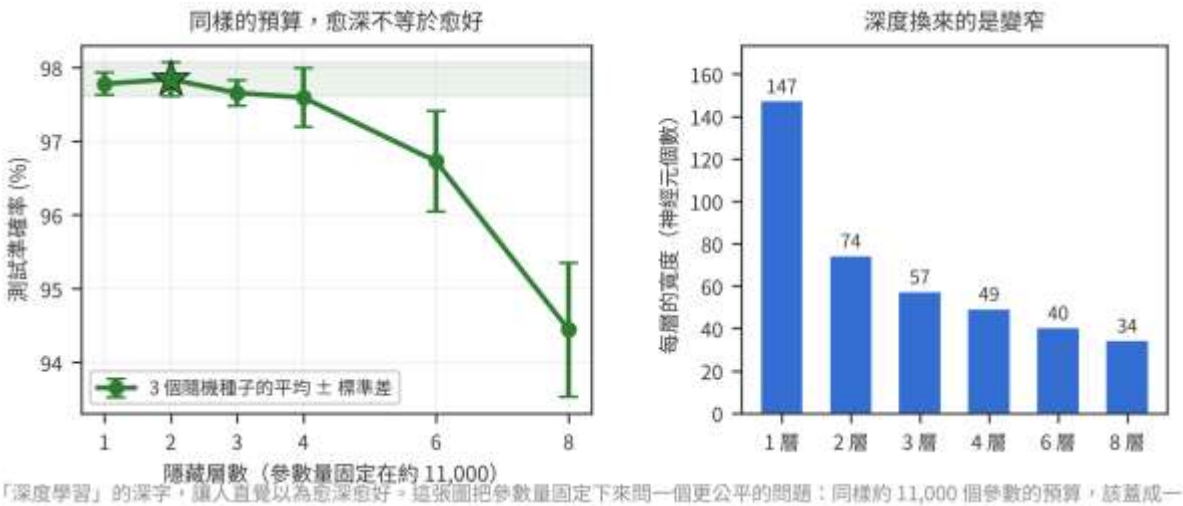


15.2.1 節說「若沒有激勵函數，無論疊多少層都等效於單一層」。這句話可以直接驗證，而且驗得乾淨俐落。左圖訓練了四個網路：一層線性拿到 96.11%，而把它加深到十層全線性（參數量多了 16 倍，10,858 個）卻只有 91.30%——不但沒變好，還更差了。原因是這十層在數學上根本是同一個模型：十個矩陣連乘仍然只是一個矩陣。本圖實際把訓練好的十層權重依序相乘摺成單一個 64×10 矩陣，再比對它與原網路對測試集的輸出，最大差異只有 1.78e-14——這個數量級就是浮點運算的捨入誤差，等於說兩者是同一個函數。既然表達能力相同，多出來的那九層就只是讓最佳化變難而已。把激勵函數加回去，一切就不同了：三層 ReLU 用 3,466 個參數拿到 97.59%，十層 ReLU 95.56%。請注意最後這個數字——十層 ReLU 反而輸給三層 ReLU，這不是打錯字，而是 15.4 節 ResNet 要解決的「退化問題」在這裡提前現身了：深度本身不會自動帶來好處，深到某個程度反而更難訓練。圖 15-16 會把這件事單獨拿出來量。

圖 15-4　把激勵函數拿掉：十層等於一層

15.2.2 多層感知器

把許多神經元排成一層、再把多層堆疊起來，就構成了多層感知器(MLP)：輸入層接收資料、若干隱藏層逐層萃取愈來愈抽象的表示、輸出層給出最終預測(例如十個輸出對應 MNIST 的十個數字)。「深度學習」的「深」，指的正是隱藏層的層數多——層數愈多，網路能表達的函數愈複雜，但也愈難訓練(這正是 AlexNet 之前深層網路難以成功的原因)。

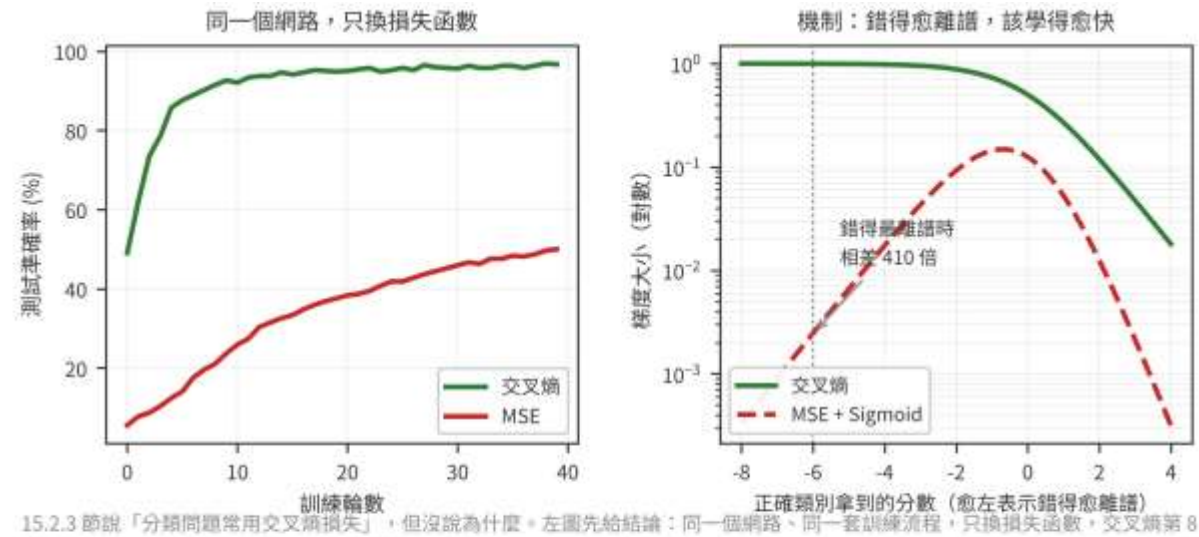


「深度學習」的深字，讓人直覺以為愈深愈好。這張圖把參數量固定下來問一個更公平的問題：同樣約 11,000 個參數的預算，該蓋成一層很寬，還是八層很窄?右圖是換算關係：一層時可以有 147 個神經元，八層時每層只剩 34 個。左圖是結果，而且每個設定都跑了三個隨機種子、附上標準差（第 14.5.3 節的規矩，沒有誤差範圍的單一數字沒有意義）。一層 97.78%、兩層 97.84%、三層 97.65%、四層 97.59%——這四個在誤差範圍內根本分不出高下;而六層掉到 96.73%、八層 94.44% ± 0.91，不但變差，種子之間的波動也明顯變大。所以在這個小任務上，深度不但沒有紅利，還開始收費。這不是說深度沒用——AlexNet 到 ResNet 的歷史正是深度的勝利——而是說深度的紅利要在「任務夠複雜、資料夠多」時才兌現得了，這與圖 15-1 的結論是同一件事。在小資料上硬疊層數，換到的往往只是更難訓練的網路。至於深到幾十層以上為什麼還能繼續變好，答案是 15.4 節的殘差連接，圖 15-16 會實際量給你看。

圖 15-5　同樣的參數量，該花在深度還是寬度

15.2.3 損失函數與梯度下降

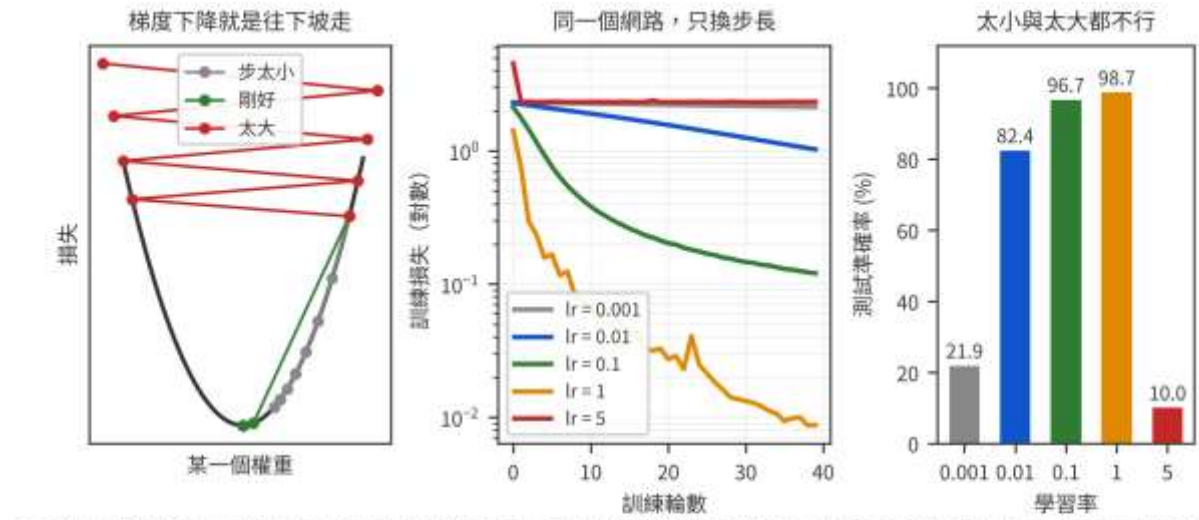
網路怎麼知道自己預測得好不好?靠損失函數(loss function)：它量化「預測」與「正確答案」之間的差距——差距愈大、損失愈高。分類問題常用交叉熵損失(cross-entropy)，迴歸問題常用均方誤差(MSE，正是第 3 章的老朋友)。訓練的目標，就是調整所有權重與偏置，讓損失盡可能小。



15.2.3 節說「分類問題常用交叉熵損失」，但沒說為什麼。左圖先給結論：同一個網路、同一套訓練流程，只換損失函數，交叉熵第 8 輪就突破 90%、最後 96.67%，而均方誤差（MSE，配 Sigmoid 輸出）四十輪只爬到 50.00%，差了 46.7 個百分點。右圖是機制，也是這張圖真正的重點。橫軸是「正確類別拿到的分數」，愈往左代表模型錯得愈離譜；縱軸是損失對該分數的梯度，也就是「這一步會被修正多少」。交叉熵的梯度在左端趨近 1：錯得愈離譜，修正力道愈大，這正是我們要的行為。MSE 配 Sigmoid 卻完全相反——Sigmoid 在兩端飽和（見圖 15-3 右圖，導數趨近 0），梯度還要再乘上這個趨近 0 的導數，於是在分數 −6 這個「錯得最離譜」的位置，交叉熵的梯度是 0.9976、MSE 只有 0.002436，相差 410 倍。換句話說，用 MSE 配 Sigmoid 做分類時，錯得最嚴重的那些樣本學得最慢——這是最糟的組合方式。交叉熵搭配 Softmax 之所以成為分類的標準配備，不是慣例，而是因為兩者的導數恰好把 Sigmoid 的飽和項約掉了。迴歸問題沒有這個飽和結構，MSE 仍然是合理的選擇。

圖 15-6　分類為什麼要用交叉熵

怎麼調?靠梯度下降(gradient descent)。想像損失是一片高低起伏的地形，而我們要走到最低點(最小損失)。梯度指出了「當前位置最陡上坡的方向」，那麼往反方向(下坡)走一小步，損失就會下降一點;反覆走，終能逼近谷底。每一步的大小由「學習率」(learning rate)控制——太大會在谷底附近來回震盪甚至衝過頭，太小則收斂慢得令人絕望，這又是一個貫穿全書的取捨。實務上用「隨機梯度下降」(SGD)及其改良版(如 Adam)，每次只用一小批(mini-batch)資料估計梯度，兼顧效率與穩定。

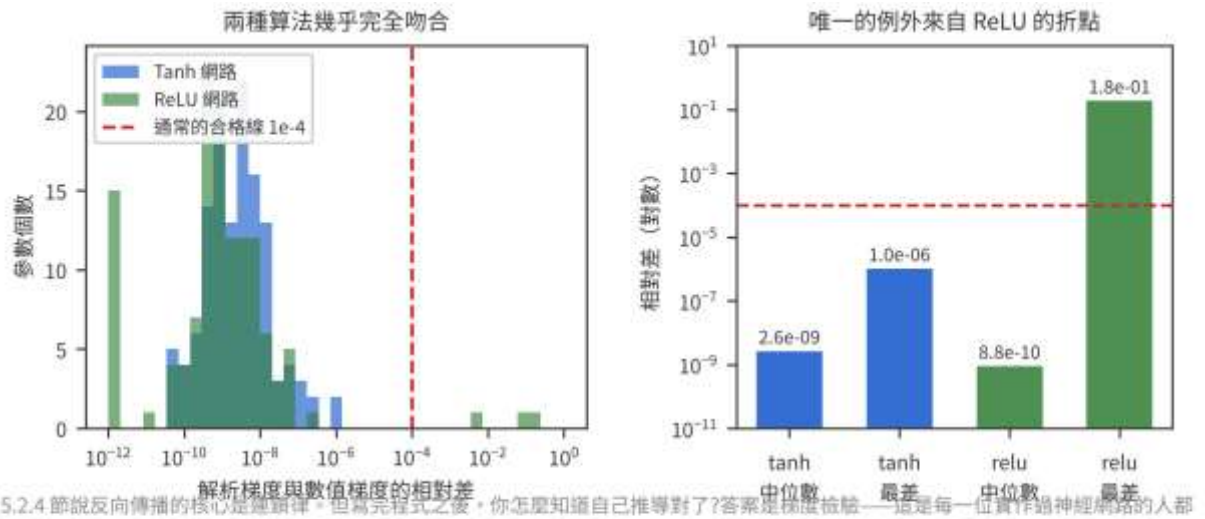


15.2.3 節把損失比喻成一片地形、梯度下降比喻成往下坡走，而學習率就是每一步跨多大。這個比喻可以量得很具體。左圖是一個簡化的碗狀損失：步子太小，走七步還在半山腰;步子剛好，幾步就到谷底;步子太大，每一步都跨過谷底、彈到對面更高的地方，愈走愈遠。中圖與右圖是同一個網路（兩層、ReLU、純 SGD）在五種學習率下的實測。lr = 0.001 時損失四十輪只從 2.32 降到 2.14，測試準確率 21.9%，幾乎等於沒學（十類亂猜是 10%）;lr = 0.01 爬到 82.4%;lr = 0.1 到 96.7%;lr = 1.0 反而最好，98.7%;而 lr = 5.0 直接發散，損失衝到 4.53 之後卡在 2.33 再也下不去，準確率 10.0%——這個數字正好就是十類亂猜。這裡有兩個該帶走的重點。其一，學習率是所有超參數裡最該優先調的那一個，因為它錯了，別的怎麼調都沒用。其二，最佳值沒辦法用背的：本例的 1.0 在多數真實任務上會炸得很難看，而這正是 Adam 這類自適應最佳化器受歡迎的原因——它替每個參數自動調整有效步長，讓人不必在這個旋鈕上耗掉太多時間。本章其餘各圖多半就是用 Adam 訓練的。

圖 15-7　學習率：走太小、剛好、與直接炸掉

15.2.4 反向傳播

最後一塊拼圖是反向傳播(backpropagation)——它回答了「如何算出每一個權重對損失的影響(梯度)」這個關鍵問題。它的核心是微積分的連鎖律：先前向計算得到預測與損失，再從輸出層往輸入層「逆向」逐層傳遞誤差，依連鎖律算出每個權重的梯度。這個演算法讓「調整深層網路裡數百萬個參數」變得可行，是神經網路能夠訓練的數學引擎。LeCun 在 1989 年正是用反向傳播訓練了 LeNet(第 14 章)，而 AlexNet 也建立在同一個基礎上。所幸，今天的深度學習框架(15.5 節)會自動完成反向傳播的所有計算，使用者無需手動推導——但理解它的原理，能幫助你診斷訓練為何失敗(如梯度消失或爆炸)。



15.2.4 節說反向傳播的核心是鏈鎖律。但寫完程式之後，你怎麼知道自己推導對了？答案是梯度檢驗——這是每一位實作過神經網路的人都該會的一招。原理很單純：梯度的定義就是「參數動一點點，損失變多少」，所以把某個權重加上與減去一個極小的 ε，各算一次損失，相減再除以 2ε，就得到了與推導完全無關的數值梯度。拿它跟反向傳播算出來的解析梯度比，兩者該吻合。本圖對一個含卷積層、池化層與全連接層的網路抽查了數百個參數：Tanh 版本的相對差中位數 2.6e-09、ReLU 版本 8.8e-10，都遠低於一般採用的合格線 1e-4——也就是說，本章後面所有數字所依賴的這份 NumPy 實作，梯度是對的。右圖那根突出的柱子則是一個值得認識的陷阱：ReLU 版本有 2.3% 的參數檢驗「失敗」，最差達到 1.8e-01。這不是程式有錯，而是 ReLU 在 0 處有折點、不可微；當某個權重剛好讓 z 落在 0 附近，加減 ε 會讓它跨過折點，此時數值梯度量到的是折點兩側的平均斜率，本來就與解析梯度不同。Tanh 處處平滑，所以完全沒有這個現象。實務上的規矩是：梯度檢驗用平滑的激勵函數做，而且看的是中位數與絕大多數參數，不是單一個最差值。

圖 15-8　反向傳播對不對：用數值梯度驗證

15.2.5 推導：連鎖律如何變成反向傳播

15.2.4 節說反向傳播的核心是微積分的連鎖律。這一小節把那句話展開成可以照著寫程式的式子。值得花時間跟一遍，理由有三個：它解釋了為什麼深層網路訓練得動、為什麼激勵函數的導數那麼要命（圖 15-9 的梯度消失）、以及為什麼 ResNet 的一個加法就能救回深度（圖 15-16）。

先把一層寫清楚。第 L 層收到前一層的輸出 a（上一層的激活值），做兩件事：先算加權和 z = a·W + b，再通過激勵函數得到 out = f(z)。整個網路就是這兩件事交替做很多次，最後把輸出送進損失函數 E。我們要的是 E 對每一個 W 與 b 的偏微分，因為梯度下降要用它來更新。

連鎖律說：若 E 透過 z 影響結果，則 E 對 W 的偏微分等於「E 對 z 的偏微分」乘上「z 對 W 的偏微分」。第二項很好算——z = a·W + b 對 W 微分就是 a。所以真正的關鍵只有第一項，我們給它一個名字，叫做這一層的誤差訊號 d：

```
d = ∂E / ∂z          （這一層加權和的誤差訊號）
```

```
∂E / ∂W = a 的轉置 · d
∂E / ∂b = d 逐列加總
```

於是問題只剩一個：怎麼算出每一層的 d。答案就是「反向」兩個字的由來——從最後一層往回推。最後一層的 d 由損失函數直接給出（Softmax 配交叉熵時，d 恰好等於「預測機率減去正確答案的 one-hot」，乾淨得不可思議，這正是 15.2.3 節那個組合的妙處，圖 15-6 量過它的後果）。而任何一層的 d，都可以由它後面那一層的 d 推出來：

```
d(本層) = ( d(後一層) · W(後一層) 的轉置 ) × f 的導數(本層的 z)
          └── 把誤差從後面傳回來 ──┘    └── 乘上本層激勵的導數 ──┘
```

這條式子就是反向傳播的全部。它說：某一層的誤差，等於把後一層的誤差透過權重「攤回來」，再乘上本層激勵函數的導數。前向計算一次把資料推到底、算出損失，反向再走一次就能拿到所有參數的梯度——總成本大約只是前向的兩倍，而不是「每個參數各試一次」那種與參數量成正比的天文數字。一個有一千萬個參數的網路，若用數值微分逐一估計梯度，每更新一步要做兩千萬次前向運算;反向傳播只要兩次。這就是神經網路能被訓練的全部理由。

現在回頭看那個「乘上本層激勵函數的導數」。每往回一層就乘一次，往回 n 層就連乘 n 次。Sigmoid 的導數最大只有 0.25（圖 15-3 右圖），所以九層之後大約只剩 0.25 的九次方，約四十萬分之一。圖 15-9 實測一個九層網路，最後一層的梯度範數是 5.09e−01，傳到第一層只剩 6.84e−06，相差約三萬七千倍——與這條算式預測的量級一致。ReLU 在正半邊的導數固定是 1，連乘多少次都不縮小，這就是它取代 Sigmoid 的數學理由。

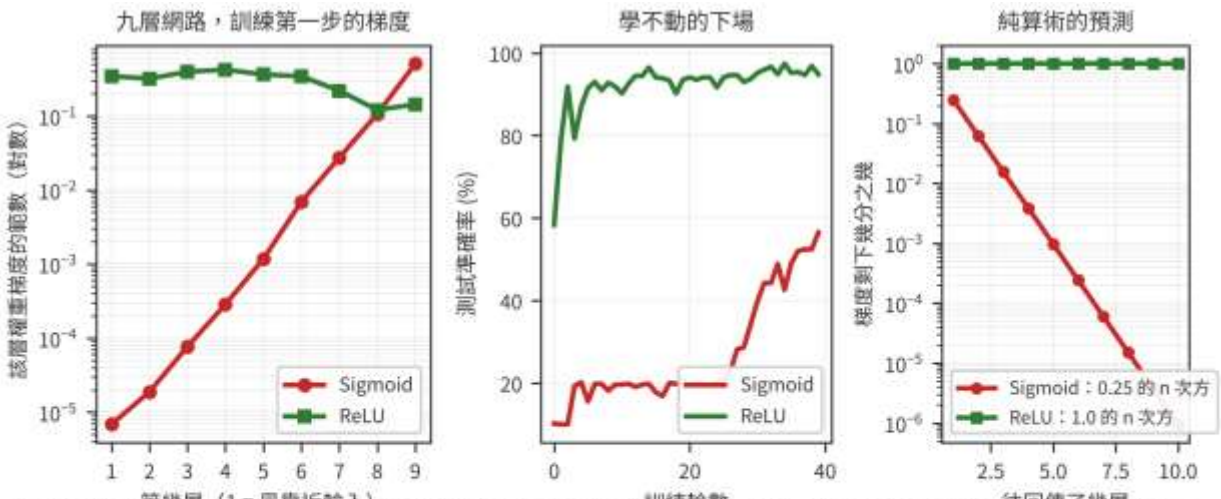


圖 15-3 右圖是「純算術」：Sigmoid 的導數最大只有 0.25，而反向傳播每往回一層就要乘它一次。0.25 連乘九次大約是 3.8e-06——右圖那條紅線就是這個預測。左圖是實測：一個九層的網路，在訓練的第一步量每一層權重的梯度範數。Sigmoid 版本最後一層的梯度是 5.09e-01，傳到第一層只剩 6.84e-06，相差 74,409 倍。這代表靠近輸入的那幾層幾乎收不到任何學習訊號，它們的權重從頭到尾跟隨機初始化差不多——說是九層，實際上只有最後兩三層在學。換成 ReLU，情況完全不同：各層梯度都在 0.12 到 0.42 之間，第一層甚至比最後一層還大（2.4 倍），訊號一路傳得到底。中圖是後果：同樣訓練四十輪，Sigmoid 版本只有 56.48%，ReLU 版本 94.81%。這就是 2012 年 AlexNet 把 ReLU 當成關鍵配方之一的原因，也是 15.4 節 ResNet 殘差連接要處理的同一個病根——當網路深到連 ReLU 都撐不住時，就需要讓梯度有一條捷徑可走（圖 15-16）。

圖 15-9　梯度消失：0.25 連乘九次會怎樣

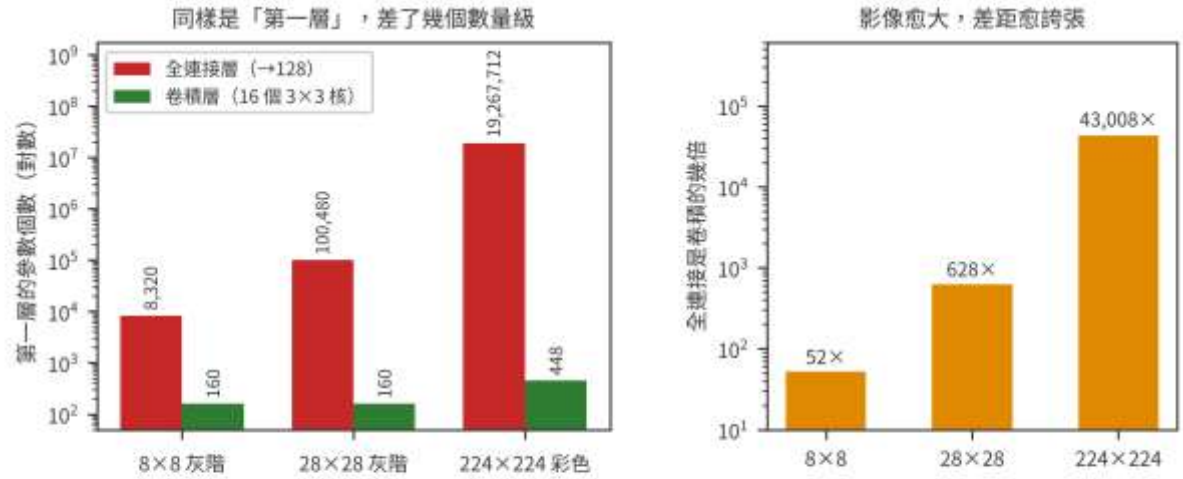
最後一個收穫是殘差連接。把一個區塊寫成 out = x + F(x)，對 x 微分會得到「1 + F 的導數」。那個 1 代表：無論 F 這條路上的導數多小，誤差訊號都有一條直通的路可以原封不動傳回去。連乘因此不再是連乘，而是「每一項都至少有 1」。圖 15-16 的實測正是這條式子的樣子：沒有捷徑的網路第一層梯度隨深度從 4.05e−01 縮到 8.73e−02，有殘差連接的反而從 6.98e−01 長到 1.27e+01。一個加號，就把梯度消失變成了梯度暢通。

實務上要補一句：框架會自動完成這一切（這叫自動微分），所以你幾乎不會親手寫這些式子。但理解它仍然重要——當訓練不動、損失變成 NaN、或深層網路怎麼調都學不起來時，能在腦中跑一遍這條式子的人，才知道該去看激勵函數、學習率、初始化還是架構。順帶一提，驗證自己推導對不對有一個標準做法：梯度檢驗，圖 15-8 示範了它，也示範了它在 ReLU 上會有的那個陷阱。

15.3 卷積神經網路(CNN)

多層感知器有一個致命問題：它把影像「攤平」成一長串像素再處理，徹底破壞

了影像的空間結構(相鄰像素的關聯性)，而且參數量爆炸——一張小小的 224×224 彩色圖攤平就是十五萬個輸入，若第一層隱藏層有一千個神經元，光第一層就要一億五千萬個權重。卷積神經網路(CNN)正是為影像量身打造，優雅地解決了這兩個問題，也是叫獸開講中 AlexNet 的骨幹。

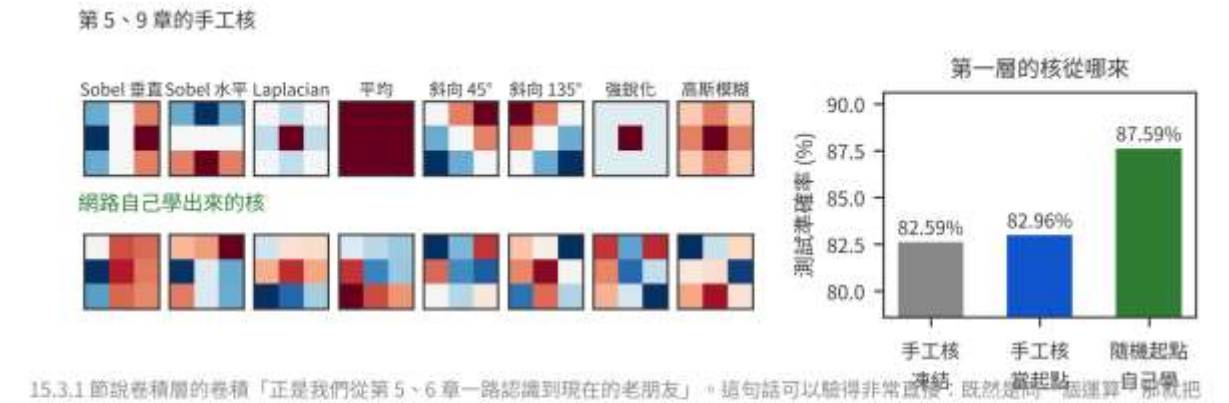


15.3 節一開頭就(本章玩具)處理影像有 MNIST 數量爆炸」(ImageNet 張圖把那個爆炸算成具體數字。一張 224×224 的彩色影像攤平之後是 150,528 個數字。若第一層就接一個 128 個神經元的全連接層，光這一層的權重就有 19,267,712 個;若比照 AlexNet 那樣接 4,096 個神經元，則是 616,562,688 個權重，用 float32 存要 2.5 GB——一層就塞不進任何一張顯示卡。而一個卷積層用 16 個 3×3×3 的核，參數只有 448 個，相差 43,008 倍。關鍵在於卷積層的參數量只跟「核多大、幾個核」有關，與影像多大完全無關——把影像換成 1024×1024，那 448 個參數一個都不會增加。右圖顯示影像愈大、差距愈誇張：8×8 時只差 52 倍，到 224×224 就是 43,008 倍。這是 15.3.1 節「權重共享」最直接的好處，但它還有第二個好處——同一組核滑遍整張影像，也意味著它在哪裡都認得同一個圖案。那一半的故事在圖 15-12。

圖 15-10　參數量爆炸：為什麼影像不能用全連接硬幹

15.3.1 卷積層：本書最熟悉的老朋友

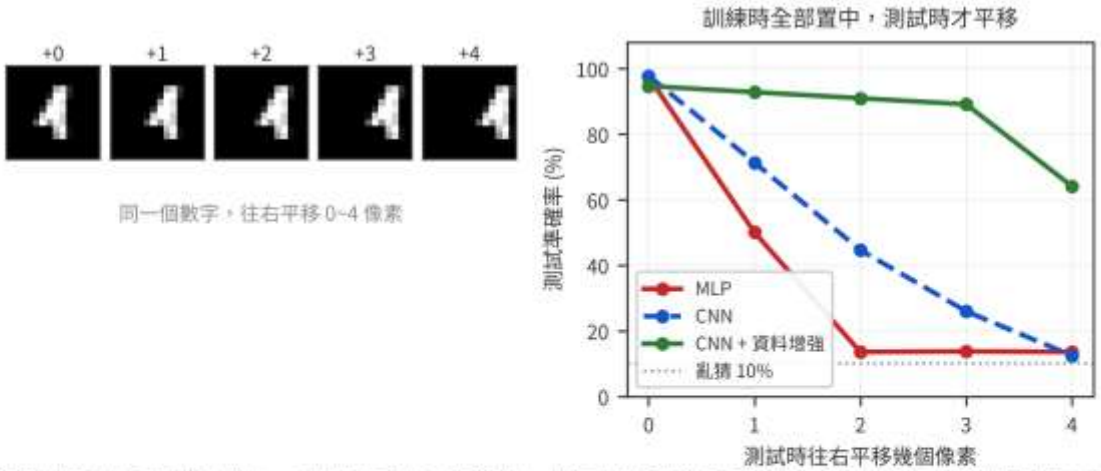
CNN 的核心是卷積層，而「卷積」正是我們從第 5、6 章一路認識到現在的老朋友!卷積層用一組小小的卷積核(filter)在影像上滑動，計算局部區域的加權和——這與第 5 章的空間濾波在數學上是同一件事。差別只有一個、卻是革命性的：第 5 章的濾波核是人設計的(高斯核、Sobel 核)，而 CNN 的卷積核是學出來的——網路透過訓練自己決定核裡的每個數值，以萃取出最有助於任務的特徵。



15.3.1 節說卷積層的卷積「正是我們從第 5、6 章一路認識到現在的老朋友」。這句話可以驗得非常直接。既然是同一個運算，那就把第 5 章的 Sobel、Laplacian、高斯模糊這些手工核，直接塞進 CNN 的第一層當卷積核，看看能跑多遠。結果是 82.59%——這些由人類在上個世紀設計出來的核，凍結不動也已經能撐起一個堪用的辨識器，說明它們確實抓到了影像裡有意義的東西。但讓網路自己從隨機初始化學起，同樣的架構拿到 87.59%，高出 5.00 個百分點。有趣的是中間那根：用手工核當起點再繼續學，只到 82.96%，幾乎沒有進步——好的起點反而把它困在附近，這是最佳化裡常見的現象。上排與下排的圖是兩種核的長相。手工核乾淨、對稱、每一個都說得出名字;學出來的核雜亂、不對稱、多半叫不出名字，卻在這個任務上更有效。這就是整個典範轉移最具體的樣子：卷積這個運算一點都沒變，變的只是「誰來決定核裡的那九個數字」。第 5 章由人決定，第 15 章由資料決定。

圖 15-11　卷積核是誰決定的：手工核 vs 學出來的核

這個設計帶來三個關鍵優勢。其一，局部連接：每個卷積核只看局部區域，符合「影像的意義來自局部結構(邊緣、紋理)」的直覺。其二，權重共享：同一個卷積核滑遍整張影像，用同一組權重——這既大幅減少參數(一個 3×3 核只有 9 個權重，不管影像多大)，又帶來對位置變化的容忍(物件出現在哪裡都能被同一個核偵測到)。這裡要把常見的「平移不變性」這個說法修得精確一點，因為它是本章最容易被過度解讀的四個字。卷積給的其實是「平移等變」——影像平移，特徵圖就跟著平移;要再加上池化的降取樣，才換到一點點「平移容忍」。圖 15-12 實測：三個模型都只用置中的數字訓練，平移 2 個像素時多層感知器崩到 13.52%(十類亂猜是 10%)、CNN 還有 44.63%，卷積確實好得多;但平移 4 個像素時 CNN 也只剩 12.41%，一樣崩了。真正的不變性要靠資料教：同一個 CNN 若訓練時就看過隨機平移的樣本(也就是 15.6.2 節的資料增強)，平移 3 個像素仍有 89.07%，是未增強版本的 3.4 倍。記住這句話：不變性多半是資料教出來的，不是架構免費送的。其三，層級特徵：淺層的卷積核學到邊緣、顏色等低階特徵，深層的則組合出紋理、部件乃至完整物件等高階特徵——這正是叫獸開講中 AlexNet「淺層學邊緣、深層認物件」的由來，也再次呼應第 6、11 章的多解析度、由粗到細的思想。

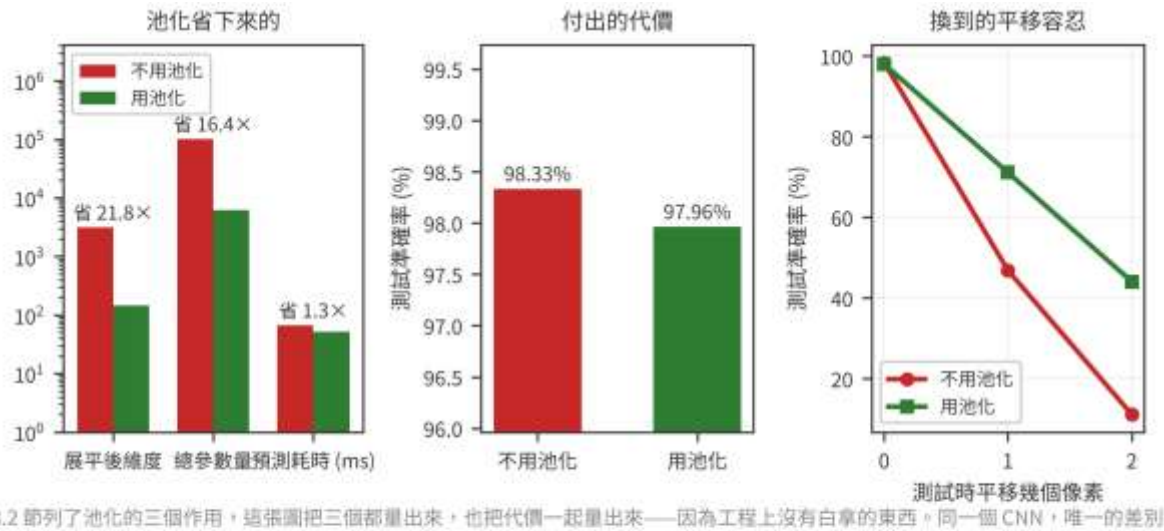


15.3.1 節說權重共享帶來「平移不變性」，這四個字在教科書裡很常見，但量過之後要修正成更精確的說法。實驗：三個模型都只用置中的數字訓練，測試時才把數字往右平移。平移 0 像素時，MLP 97.41%、CNN 97.78%，不相上下。平移 2 個像素，MLP 崩到 13.52%（十類亂猜是 10%），CNN 還有 44.63%——確實好得多，卷積的權重共享真的有用。但請看平移 4 個像素：CNN 也只剩 12.41%，一樣崩了。所以嚴格說，卷積給的是「平移等變」（影像平移，特徵圖跟著平移），加上池化之後換到一點點「平移容忍」，但距離真正的「不變」還很遠。第三條線是補上的那一塊：同一個 CNN，訓練時就讓它看過隨機平移過的樣本（也就是 15.6.2 節的資料增強，用的是第 4 章的幾何轉換）。它在平移 3 個像素時仍有 89.07%，是未增強版本 25.93% 的 3.4 倍。代價是置中時略低一點（94.63% vs 97.78%），因為同樣的容量要分給更多種情況。結論值得記住：不變性不是架構免費送的，多半是資料教出來的。

圖 15-12　權重共享給的是容忍，不是不變

15.3.2 池化層

池化層(pooling)負責「降取樣」——縮小特徵圖的尺寸。最常見的最大池化(max pooling)把每個小區域(如 2×2)只保留最大值，尺寸減半。它有三個作用：減少後續運算量與參數、擴大後續卷積核的「視野」(讓深層能看到更大範圍)、並提供一定的位置容錯(物件稍微移動，池化結果變化不大)。有趣的是，最大池化與最小池化，本質上就是第 10 章的型態學膨脹與侵蝕——又一次一體多面的印證。

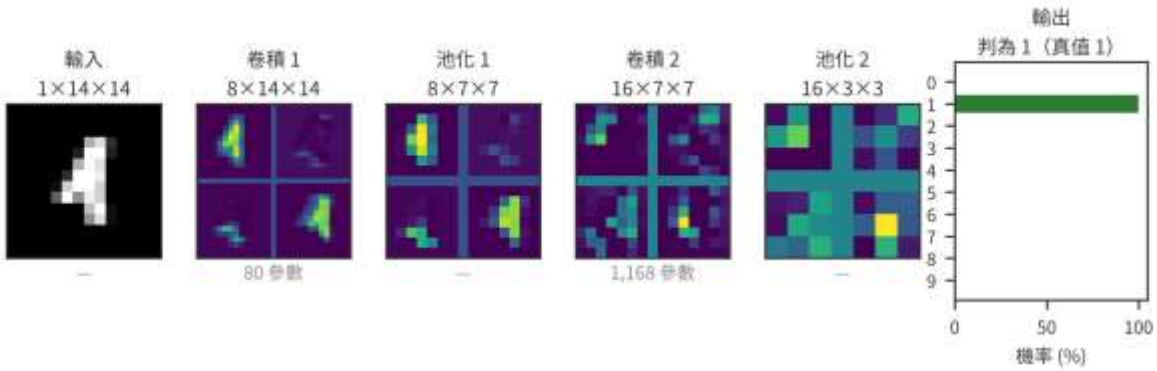


15.3.2 節列了池化的三個作用，這張圖把三個都量出來，也把代價一起量出來——因為工程上沒有白拿的東西。同一個 CNN，唯一的差別是拿掉兩個池化層。拿掉之後，卷積的輸出仍是 14×14，展平後是 3,136 維，接上全連接層使總參數量達到 101,962 個;加回池化，展平後只剩 144 維、總參數 6,218 個，足足少了 16.4 倍，預測也快了 1.3 倍。第二個作用是擴大感受野：3×3 卷積的感受野是 3 個像素，經過一次 2×2 池化之後，下一個 3×3 卷積看到的已經相當於原圖的 10 個像素，再池化一次就是 20 個——這就是「層級特徵」能從邊緣長成形狀再長成物件的原因。第三個作用是平移容忍：右圖顯示平移 2 個像素時，有池化的 44.1%、沒池化的只有 11.1%，差了 4.0 倍。代價寫在中圖：準確率從 98.33% 降到 97.96%，少了 0.37 個百分點。用 0.37 個百分點換 16 倍的參數、1.3 倍的速度與 4.0 倍的平移容忍，這筆交易在幾乎任何實務場合都划算——而這也正是第 17 章要把模型塞進邊緣裝置時最先想到的那類取捨。

圖 15-13　池化層用零點幾個百分點換到了什麼

15.3.3 全連接層與整體結構

一個典型的 CNN，是「卷積層 + 池化層」的多次堆疊(負責萃取由低階到高階的特徵)，最後接上幾層全連接層(第 15.2 節的 MLP，負責把萃取到的特徵映射到最終的類別)。這個「卷積池化萃取特徵、全連接做決策」的結構，從 1998 年的 LeNet 到 2012 年的 AlexNet 一脈相承，也是下一節架構譜系的共同骨架。



15.3.3 節描述的「卷積-池化多次堆疊，最後接全連接層」，這張圖把一個真實的小 CNN 從頭到尾走一遍（測試準確率 97.78%，總共 6,218 個參數）。請同時看三件事。第一是尺寸：14×14 的輸入經過卷積後空間大小不變、但變成 8 張特徵圖;每經過一次池化，長寬就減半（14→7→3），而圖的張數持續增加（8→16）。這個「空間變小、通道變多」的形狀，是幾乎所有 CNN 的共同輪廓——它在做的事就是把「哪裡有什麼」逐步換成「有什麼」。第二是內容：卷積 1 的特徵圖還看得出數字的輪廓與筆畫，到了池化 2 的 3×3 已經完全看不出原來的字形，只剩下一些抽象的激活強度——這正是「由低階到高階特徵」的具體樣貌。第三是參數花在哪：兩個卷積層加起來只有 1,248 個參數，而最後兩個全連接層就用掉 4,970 個，佔了總數的 80%。這個比例在真實架構裡更誇張（VGG-16 有九成參數在全連接層），也是 15.4 節 GoogLeNet 與 MobileNet 要用全域平均池化取代大型全連接層的動機。最右邊是輸出：十個類別的機率，取最大的那個當答案。

圖 15-14　走完一次 CNN：每一層看到什麼、花了多少參數

15.3.4 推導：參數量、感受野與那個「空間變小、通道變多」的形狀

15.3 節用文字說了卷積層省參數、池化層擴大感受野。這兩件事都有簡單的公式，而且算一遍之後，你會發現幾乎所有 CNN 長得像同一個形狀，不是巧合。

先算參數量。一個卷積層若輸入有 Cin 個通道、輸出 Cout 個通道、卷積核是 k×k，則參數量是：

```
卷積層參數 = k × k × Cin × Cout + Cout        （最後一項是偏置）
全連接層參數 = (H × W × Cin) × Cout + Cout
```

關鍵差別一眼可見：卷積層的式子裡沒有 H 與 W。影像多大，卷積層的參數量都一樣——因為同一組核滑遍全圖，這就是「權重共享」的數學形式。圖 15-10 用真實尺寸算過：224×224 的彩色影像攤平是 150,528 個數字，接一個 128 個神經元的全連接層要 19,267,712 個參數;而 16 個 3×3×3 的卷積核只要 448 個，相差 43,008 倍。把影像換成 1024×1024，前者會再膨脹二十倍，後者一個參數都不變。

再算感受野，也就是「輸出上的一個點，看得到原圖多大一塊」。它有一條遞迴式：

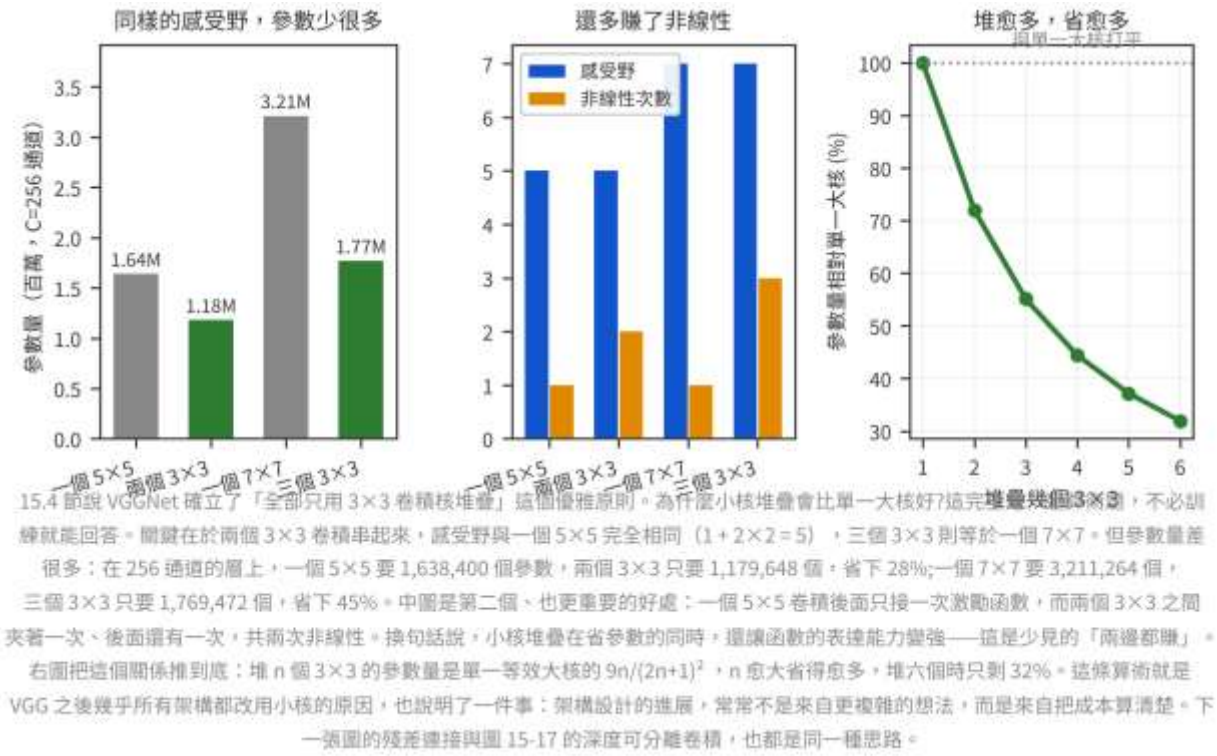
```
RF(新) = RF(舊) + (k − 1) × 跳躍步幅(舊)
```

```
跳躍步幅(新) = 跳躍步幅(舊) × stride
```

代進一個典型的堆疊就很清楚。一開始感受野 1、步幅 1;經過 3×3 卷積，感受野變 3;經過 2×2 池化，感受野變 4、而步幅變成 2;再一個 3×3 卷積，感受野變成 4 + 2×2 = 8;再池化一次，感受野 10、步幅 4。換句話說，每加一次池化，後面每一層的「視野成長速度」就翻一倍。這是池化最被低估的作用：它不只是省計算，更是讓深層能夠「看見整體」的手段。沒有降取樣的話，想讓感受野涵蓋整張 224×224 的影像，用 3×3 卷積要疊一百多層;有了五次池化，十幾層就夠了。

這兩條式子合起來，就解釋了 CNN 那個標準形狀：空間尺寸一路減半（224→112→56→28→14→7），通道數一路加倍（64→128→256→512）。為什麼要同步加倍?因為每次池化都讓特徵圖的面積變成四分之一，若通道數不變，這一層能攜帶的資訊量就掉到四分之一;把通道加倍，總量只掉到一半，比較不會形成瓶頸。同時，深層的特徵愈來愈抽象、需要的種類愈來愈多（淺層只需要少數幾種邊緣，深層要分辨幾百種物件部件），所以通道變多也是語意上的需要。圖 15-14 的小 CNN 走的正是這個形狀：1×14×14 → 8×14×14 → 8×7×7 → 16×7×7 → 16×3×3。

最後一個算術解釋了 15.4 節的兩個架構決策。其一，兩個 3×3 卷積串起來的感受野是 1 + 2×2 = 5，與一個 5×5 相同，但參數量是 2×9 = 18 對 25，只有 72%，而且多了一次非線性——這就是 VGG 全面改用 3×3 的理由（圖 15-15）。其二，標準卷積的 k×k×Cin×Cout 可以拆成「每通道各做一次 k×k」加上「1×1 混合通道」，參數變成 k×k×Cin + Cin×Cout，兩者的比值是 1/Cout + 1/k²，通道數夠多時大約就是 1/k²，也就是省下約 k² 倍——這就是 MobileNet 的深度可分離卷積（圖 15-17）。架構設計看起來像藝術，底層往往就是這幾條式子在算成本。



15.4 節說 VGGNet 確立了「全部只用 3×3 卷積核堆疊」這個優雅原則。為什麼小核堆疊會比單一大核好？這完全不必訓練就能回答。關鍵在於兩個 3×3 卷積串起來，感受野與一個 5×5 完全相同（1 + 2×2 = 5），三個 3×3 則等於一個 7×7。但參數量差很多：在 256 通道的層上，一個 5×5 要 1,638,400 個參數，兩個 3×3 只要 1,179,648 個，省下 28%；一個 7×7 要 3,211,264 個，三個 3×3 只要 1,769,472 個，省下 45%。中圖是第二個、也更重要的好處：一個 5×5 卷積後面只接一次激勵函數，而兩個 3×3 之間夾著一次、後面還有一次，共兩次非線性。換句話說，小核堆疊在省參數的同時，還讓函數的表達能力變強——這是少見的「兩邊都賺」。右圖把這個關係推到底：堆 n 個 3×3 的參數量是單一等效大核的 $9n/(2n+1)^2$，n 愈大省得愈多，堆六個時只剩 32%。這條算術就是 VGG 之後幾乎所有架構都改用小核的原因，也說明了一件事：架構設計的進展，常常不是來自更複雜的想法，而是來自把成本算清楚。下一張圖的殘差連接與圖 15-17 的深度可分離卷積，也都是同一種思路。

圖 15-15　VGG 的算術：為什麼全部改用 3×3

15.4 經典 CNN 架構譜系

自 AlexNet 引爆革命後，CNN 架構快速演進，每一代都針對前一代的瓶頸提出突破。理解這條譜系，就是理解深度學習視覺這十餘年的發展主線。

1. LeNet(1998,LeCun):CNN 的開山始祖，約七層，用於 MNIST 手寫數字辨識(第 14 章)。它奠定了「卷積-池化-全連接」的基本範式，但受限於當年的資料與算力，未能大放異彩。
2. AlexNet(2012,Krizhevsky 等)：叫獸開講的主角，八層。以 ReLU、Dropout、GPU 訓練與資料增強，在 ImageNet 上一舉成名，引爆深度學習革命。它證明了「夠深的網路 + 大數據 + GPU」的威力。
3. VGGNet(2014，牛津)：把網路加深到 16、19 層，並確立了一個優雅的原則——全部只用小小的 3×3 卷積核堆疊。它證明了「深度」本身的價值，結構規整、易於理解，至今仍是教學與遷移學習的常用骨幹。

4. GoogLeNet / Inception(2014,Google)：同年冠軍，提出「Inception 模組」——在同一層並行使用多種尺寸的卷積核(1×1、3×3、5×5)，讓網路自己選擇適合的尺度(呼應多尺度思想);並用 1×1 卷積大幅壓縮參數，以 22 層的深度卻比 AlexNet 更省參數。

5. ResNet(2015，微軟何愷明等)：解決了一個致命瓶頸——網路太深時，梯度難以傳遞、反而更難訓練(退化問題)。它的神來之筆是「殘差連接」(skip connection)：讓資訊可以「跳過」若干層直接往後傳，使得訓練上百層的網路成為可能。ResNet 在 ImageNet 上的錯誤率首度低於人類水準(約 5%)，是深度學習史上又一里程碑，殘差連接的思想至今無所不在(包括 Transformer)。

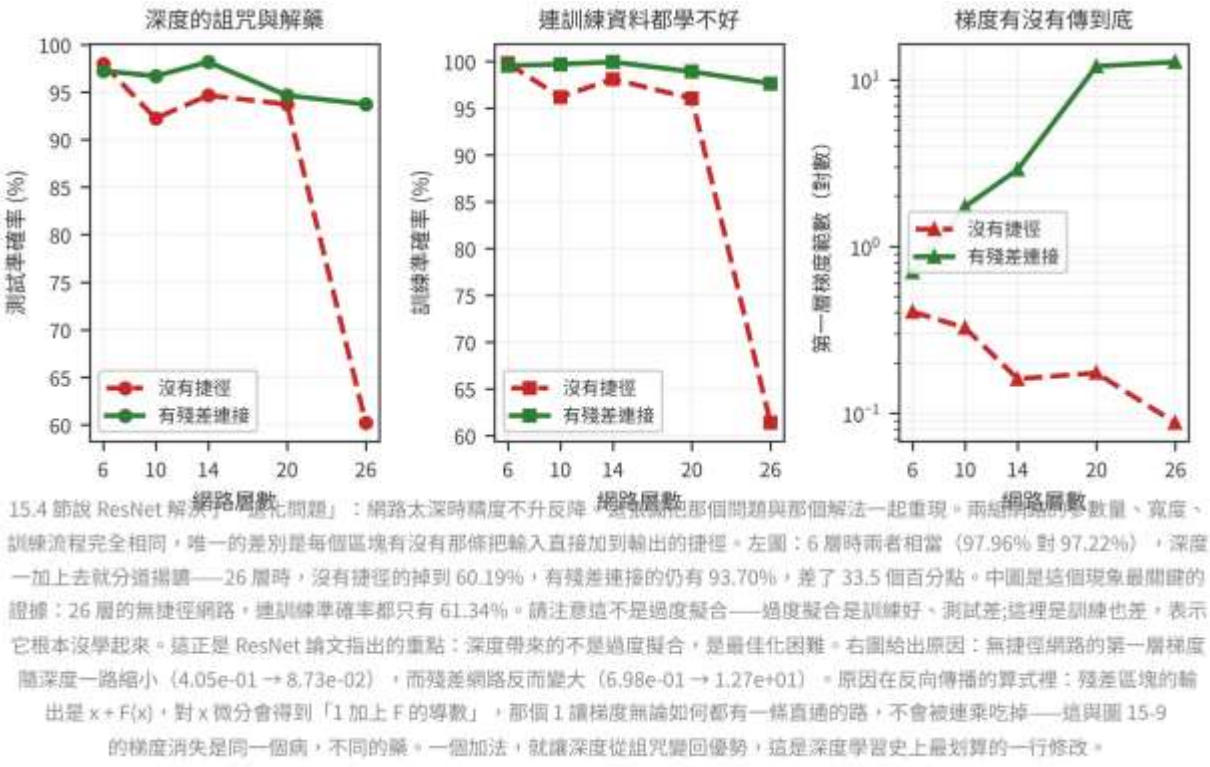
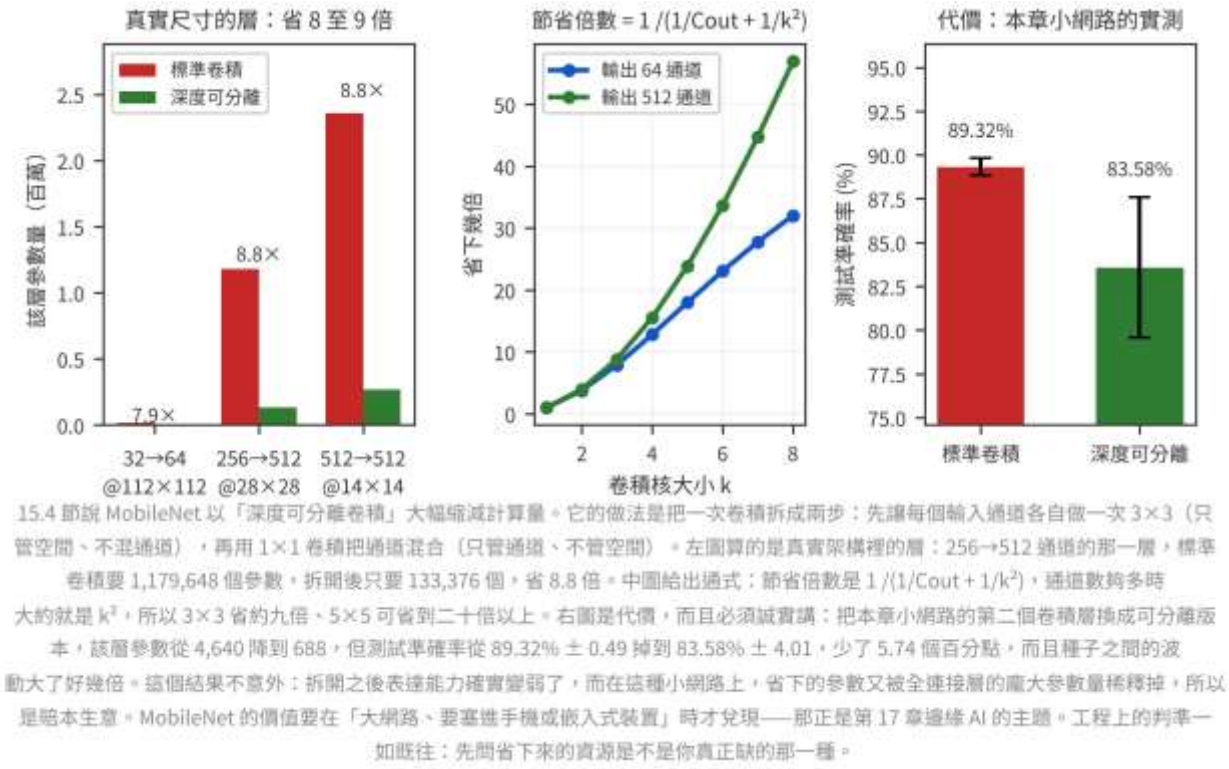


圖 15-16　殘差連接：讓深度重新變成優勢

6. MobileNet(2017,Google)：前面的架構愈做愈深、愈準，但也愈龐大、愈耗算力，無法在手機或嵌入式裝置上執行。MobileNet 反其道而行，以「深度可分離卷積」大幅壓縮運算量與模型大小，在可接受的精度損失下，讓 CNN 能在手機、樹莓派、Jetson 等資源受限的邊緣裝置上即時執行——這正是第 17 章邊緣部署的關鍵，也呼應第 9、10、13 章反覆強調的「適合勝於最強」。

這條譜系揭示了一個清晰的張力：一邊是「愈深愈準」的極致追求(VGG、ResNet)，一邊是「夠好且能部署」的務實考量(MobileNet)。成熟的工程師懂得依場景選擇——雲端伺服器追求極致精度，邊緣裝置追求效率，沒有一個架構適用於所有場合。這也是為什麼您的教材要完整涵蓋整條譜系：它們不是「新的取代舊的」，而是各自佔據不同的應用生態位。



15.4 節說 MobileNet 以「深度可分離卷積」大幅縮減計算量。它的做法是把一次卷積拆成兩步：先讓每個輸入通道各自做一次 3×3（只管空間、不混通道），再用 1×1 卷積把通道混合（只管通道、不管空間）。左圖算的是真實架構裡的層：256→512 通道的那一層，標準卷積要 1,179,648 個參數，拆開後只要 133,376 個，省 8.8 倍。中圖給出通式：節省倍數是 $1/(1/\text{Cout} + 1/k^2)$，通道數夠多時大約就是 k^2，所以 3×3 省約九倍、5×5 可省到二十倍以上。右圖是代價，而且必須誠實講：把本章小網路的第二個卷積層換成可分離版本，該層參數從 4,640 降到 688，但測試準確率從 89.32% ± 0.49 掉到 83.58% ± 4.01，少了 5.74 個百分點，而且種子之間的波動大了好幾倍。這個結果不意外：拆開之後表達能力確實變弱了，而在這種小網路上，省下的參數又被全連接層的龐大參數量稀釋掉，所以是賠本生意。MobileNet 的價值要在「大網路、要塞進手機或嵌入式裝置」時才兌現——那正是第 17 章邊緣 AI 的主題。工程上的判準一如既往：先問省下來的資源是不是你真正缺的那一種。

圖 15-17　深度可分離卷積：MobileNet 的那筆帳

表 15-1 經典 CNN 架構譜系

15.5 三大框架與 MNIST 實作

手動實作神經網路(尤其是反向傳播)極為繁瑣，所幸有深度學習框架代勞——它們自動處理梯度計算、GPU 加速、以及大量現成的網路元件。三大主流框架各有特色：

1. TensorFlow(Google,2015)：工業界部署的重鎮，生態完整，尤其擅長大規模生產環境與行動端(TensorFlow Lite 可部署到手機，更受限的微控制器則有 TensorFlow Lite for Microcontrollers，呼應第 17 章)。

2. Keras：原為獨立的高階 API、現已整合進 TensorFlow，以極簡潔的語法著稱，幾行程式就能搭出一個網路，是初學者與快速原型的首選——也是您課程 MNIST 教學的理想起點。
3. PyTorch(Meta,2016)：學術研究的主流，以直觀的「動態計算圖」與貼近 Python 的風格深受研究者喜愛，除錯容易，近年在產業界也快速普及。

您的教材採用「TensorFlow → Keras → PyTorch」的跨框架 MNIST 實作，用意深遠：同一個手寫數字分類任務，用三種框架各做一遍，學生能清楚看見「網路架構的思想是共通的，只是語法不同」——這培養的是不被單一工具綁架的核心理解力。以下以 Keras 示範最簡潔的版本。

程式 15-1 Keras 實作 MNIST CNN 分類器

```
import tensorflow as tf
from tensorflow import keras
from tensorflow.keras import layers

# 1. 載入 MNIST(第 14 章的經典資料集)
(x_train, y_train), (x_test, y_test) = keras.datasets.mnist.load_data()
x_train = x_train.reshape(-1, 28, 28, 1) / 255.0   # 正規化到 0~1
x_test  = x_test.reshape(-1, 28, 28, 1) / 255.0

# 2. 建構 CNN(卷積-池化-全連接,15.3 節的結構)
model = keras.Sequential([
    layers.Conv2D(32, 3, activation="relu", input_shape=(28,28,1)),
    layers.MaxPooling2D(),                          # 池化降取樣
    layers.Conv2D(64, 3, activation="relu"),
    layers.MaxPooling2D(),
    layers.Flatten(),                               # 攤平接全連接
    layers.Dropout(0.5),                            # 對抗過度擬合(第 14 章)
    layers.Dense(10, activation="softmax")          # 10 類輸出
])

# 3. 編譯:指定損失函數與最佳化器(15.2 節)
model.compile(optimizer="adam",
              loss="sparse_categorical_crossentropy",
              metrics=["accuracy"])

# 4. 訓練
model.fit(x_train, y_train, epochs=5, batch_size=128,
          validation_split=0.1)     # 切出驗證集(第 14 章)

# 5. 評估(用沒看過的測試集,第 14 章)
loss, acc = model.evaluate(x_test, y_test)
```

```
print(f"測試準確率:{acc:.4f}")      # 通常可達 99% 以上
```

請留意這段程式如何綜合了前兩章的觀念：正規化(第 3 章)、卷積與池化(本章)、Dropout 對抗過度擬合(第 14 章)、切驗證集與用測試集評估(第 14 章)、交叉熵損失與 Adam 最佳化器(本章)。短短二十行，是全書多章觀念的結晶。

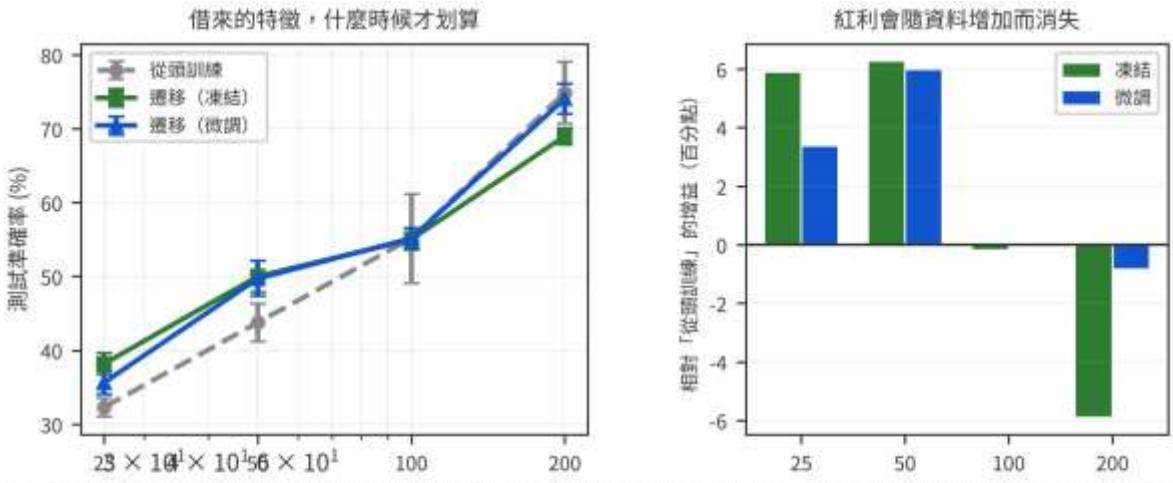
15.6 遷移學習與資料增強

15.6.1 遷移學習：站在巨人的肩膀上

訓練一個像 ResNet 這樣的大型網路，需要 ImageNet 那樣的百萬級資料與大量 GPU 時間——這對絕大多數應用(尤其是資料稀少的中小企業與學生專題)是不切實際的。遷移學習(Transfer Learning)是這個困境的救星，其思想極為漂亮：一個在 ImageNet 上訓練好的網路，它較低的層已經學會了通用的視覺特徵(邊緣、紋理、形狀)——這些特徵對任何影像任務都有用，不必從頭再學。

因此，我們可以「借用」一個預訓練模型：保留它已經學好的特徵萃取層(凍結權重)，只替換並重新訓練最後負責分類的幾層，使其適應我們自己的任務。這樣一來，即使只有幾百張自己的影像，也能訓練出高效能的模型——因為困難的「學特徵」部分已經由巨人(ImageNet 預訓練)完成了，我們只需教會它「把這些特徵對應到我的類別」。這呼應第 14 章「資料是燃料」：遷移學習讓你借用別人燒過的燃料所產生的成果。對雲林在地的農業分級、工業檢測等樣本有限的應用，遷移學習往往是比從頭訓練更務實、更高效的選擇。但這句話有一個必須補上的條件，否則很容易用錯。圖 15-18 把「凍結式遷移」「從頭訓練」「小學習率微調」三條路線放在同一個實驗裡比較：目標任務每類只有 5 張(共 25 張)時，凍結式遷移 38.17%、從頭訓練 32.29%，遷移領先 5.9 個百分點;但每類增加到 40 張(共 200 張)時，凍結式 68.97% 反而輸給從頭訓練的 74.85%，落後 5.9 個百分點。道理不難理解：凍結等於宣告「這些特徵不准改」，資料少時這是有益的約束，資料多到足以自己學出更貼合的特徵時，它就變成了天花板(用第 14.6.4 節的話說，就是用偏差換變異，而這筆交易會隨資料量改變划不

划算)。實務上的標準答案因此是微調：不凍結，但用較小的學習率更新整個網路，讓預訓練權重當成好的起點而非硬性約束;實測它在小資料時接近凍結式、資料變多時又追上從頭訓練，兩邊都不吃虧。一句話：資料愈少，凍結愈划算;資料變多，就該把特徵層解凍——而交叉點在哪裡，只能靠驗證集找。

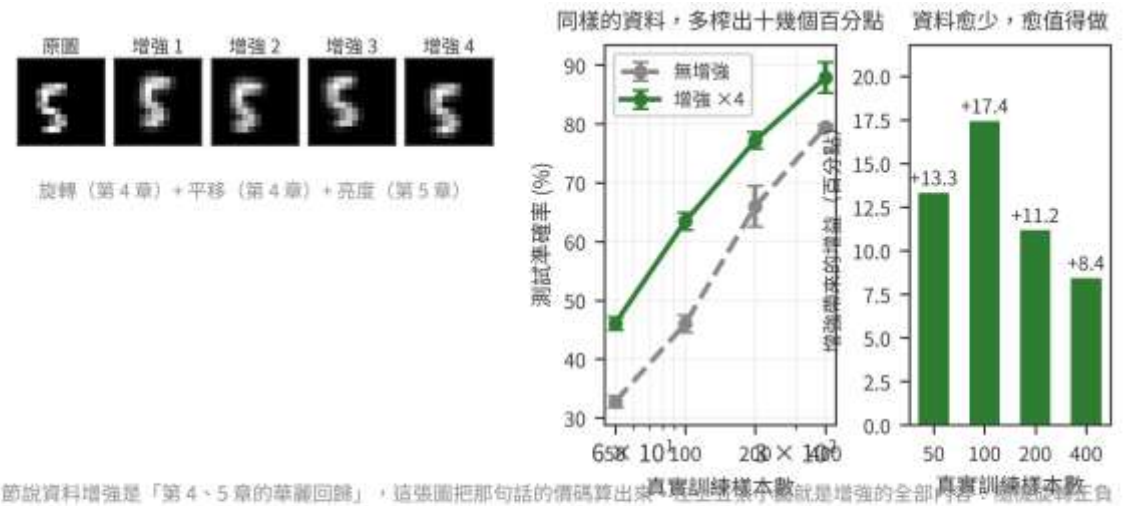


15.6.1 節說遷移學習是「借來的特徵」，這張圖用一個乾淨的設計把這句話量出來，並補上它沒說的那一半。來源任務是數字 0 到 4（901 張，預訓練到訓練準確率 100%），目標任務是完全沒看過的數字 5 到 9，樣本數從 25 張變化到 200 張。小資料時遷移學習確實有效：每類 5 張（共 25 張）時，凍結式遷移 38.17%、從頭訓練只有 32.29%，多了 5.88 個百分點;每類 10 張時仍多 6.25 個百分點。但請看右邊兩根：每類 40 張（共 200 張）時，凍結式遷移 68.97% 反而輸給從頭訓練的 74.85%，落後 5.88 個百分點。道理不難理解：凍結等於宣告「這些特徵不准改」，資料少時這是有益的約束，資料多到足以自己學出更貼合的特徵時，它就變成了天花板。第三條線是實務上的標準答案——不凍結、用較小的學習率微調整個網路。它在小資料時接近凍結式（35.64%），資料變多時又追上從頭訓練（74.03%），兩邊都不吃虧。所以 15.6.1 節那句話要補一個條件：資料愈少，凍結愈划算;資料變多，就該把特徵層解凍。這個判斷點只能靠驗證集找（第 14.5 節）。

圖 15-18　遷移學習：什麼時候該借，什麼時候該自己學

15.6.2 資料增強：第 4、5 章的華麗回歸

第 14 章介紹過，資料增強是對抗過度擬合最有效的手段之一，而它用的正是本書前面的技術!對訓練影像做隨機的旋轉、平移、翻轉(第 4 章幾何轉換)、亮度與對比調整(第 5 章空間域強化)、乃至加入雜訊(第 7 章)——每一種變換都產生一張「新」的訓練樣本，人工擴充了資料的多樣性，逼迫模型學習真正穩健的特徵(而非死背某張圖的細節)。



15.6.2 節說資料增強是「第 4、5 章的華麗回歸」，這張圖把那句話的價碼算出來——這整張圖就是增強的全部內容。橫軸是真實訓練樣本數，縱軸是測試準確率。左圖：藍線是沒有增強的 CNN，綠線是加上增強後的 CNN，兩條線之間的垂直距離就是增強帶來的準確率提升（百分點）。右圖：把這段距離單獨畫成長條，數字標在每根條上。增強的內容是隨機旋轉正負 12 度（第 4 章的幾何轉換）、隨機平移一個多像素（第 4 章）、亮度乘上 0.8 到 1.2 倍（第 5 章的影像強化）。沒有任何新資料、沒有任何新標註，只是把同一張圖換個樣子再看一次。右圖是效果：只有 50 張真實樣本時，不增強的 CNN 只有 32.73%，增強到四倍之後升到 46.04%，多了 13.3 個百分點;100 張時更誇張，增益達到 17.4 個百分點。但請注意最右邊那根：400 張時增益縮到 8.4 個百分點。資料愈多，增強的邊際效益愈小——這與第 14.6.3 節圖 14-19 量到的趨勢完全一致，因為兩者對抗的是同一個東西：變異。這裡值得停下來想一件事：增強用的全是本書第 4、5 章的基本運算，沒有任何深度學習的成分，卻是小資料情境下性價比最高的一招——它便宜、不需要標註、而且立刻見效。代價是必須選對增強方式：對手寫數字做上下翻轉會把 6 變成 9，那不是增強而是加錯標籤。增強要保留「標籤不變」這個前提，這需要的是領域知識，不是演算法。

圖 15-19　資料增強：第 4、5 章技術的華麗回歸

叫獸開講中的 AlexNet 正是資料增強的早期實踐者——它用隨機裁切、水平翻轉、色彩擾動來擴充 ImageNet，顯著降低了過度擬合。這個美妙的連結值得再次強調：深度學習訓練中不可或缺的資料增強，骨子裡就是第 4、5、7 章的影像處理技術。這再次證明，深度學習不是憑空而來的新學問，而是穩穩地站在傳統影像處理的地基之上——這正是本教科書把深度學習放在最後、而非一開頭的深意。

15.6.3 推導：遷移學習與資料增強，為什麼在小資料上特別有效

15.6 節介紹了兩個對付小資料的手段，但沒說它們為什麼有效、又在什麼時候會失效。答案藏在第 14.6.4 節推導過的偏差-變異分解裡，把它拿來重讀一次，兩件事就都說得通了。

回憶那條等式：總誤差 = 偏差² + 變異 + 雜訊。深度學習模型的參數動輒數百萬，表達能力極強，所以偏差很低;但也因此對訓練資料極度敏感，變異很高。換一批訓練資料就學出一個差很多的模型——這正是小資料情境下深度學習最大的敵人。所以對付小資料的手段，目標幾乎都是同一個：壓低變異。

資料增強是最直接的一招。它利用一個領域知識：對影像做小幅旋轉、平移、亮度調整之後，標籤不會改變。於是一張標註可以生出許多訓練樣本，等於告訴模型「這些變化都不該影響你的答案」，把原本要靠資料量才能學到的不變性，直接寫進訓練集裡。圖 15-19 實測：只有 50 張真實樣本時，增強到四倍讓準確率從 32.73% 升到 46.04%，多了 13.3 個百分點;但 400 張時增益縮到 8.4 個百分點。這個遞減完全符合預期——增強出來的樣本彼此高度相關，提供的「新資訊」遠少於等量的真實資料，當真實資料已經夠多，它能壓的變異就所剩無幾了。第 14.6.3 節圖 14-19 在傳統模型上量到的也是同一個趨勢。

這裡要補一個常被忽略的前提：增強必須保持標籤不變。對手寫數字做上下翻轉會把 6 變成 9，那不是增強，是在訓練集裡加錯誤標籤;對醫學影像做左右翻轉可能把心臟換到右邊，造出不存在的病例。選擇增強方式需要的是領域知識，不是演算法——這也是為什麼自動化增強搜尋至今仍是研究題目。

遷移學習壓低變異的方式更強硬：它直接凍結大部分參數。一個凍結了特徵層的模型，可訓練參數可能只剩原來的百分之幾，變異自然大幅下降。它之所以行得通，是因為低層特徵（邊緣、紋理、顏色漸變）對幾乎所有視覺任務都通用——這一點與第 6 章的 Gabor、第 13 章的 SIFT/HOG 是同一個洞見，只是這次的通用特徵是從一百萬張影像上學出來的，而不是人設計的。

但凍結同時也把偏差鎖住了。被凍結的特徵層學不到目標任務特有的東西，這在資料少時無所謂（反正也學不準），資料一多就變成天花板。圖 15-18 把這個交叉點量了出來：每類 5 張時，凍結式遷移 38.17%、從頭訓練 32.29%，遷移領先 5.9 個百分點;但每類 40 張時，凍結式 68.97% 反而輸給從頭訓練的 74.85%，落後 5.9 個百分點。用偏差-變異的話來說：資料少時「少一點變異」值得用「多一點偏差」去換，資料多時這筆交易就不划算了。

實務上的標準做法因此是微調（fine-tuning）：不凍結，但用比較小的學習率更

新整個網路，讓預訓練權重當成一個好的起點而非硬性約束。圖 15-18 的第三條線顯示它兩邊都不吃虧——小資料時接近凍結式，資料變多時追上從頭訓練。常見的折衷還有「分段解凍」：先只訓練分類頭，等它穩定了再解凍高層、最後解凍低層，每解凍一次就把學習率調小一些。

最後把這三章串起來。第 14 章給了偏差-變異這把尺;第 15 章的遷移學習與資料增強，是這把尺在深度學習上的兩個具體用法;而該用哪一個、用到什麼程度，答案永遠來自驗證集，不是來自預設值。深度學習換掉了特徵工程，但沒有換掉第 14 章任何一條評估規矩。

程式 15-2 遷移學習與資料增強

```
from tensorflow import keras
from tensorflow.keras import layers

# --- 資料增強:全是第 4、5、7 章的影像處理技術 ---
augment = keras.Sequential([
    layers.RandomFlip("horizontal"),      # 第 4 章:翻轉
    layers.RandomRotation(0.1),           # 第 4 章:旋轉
    layers.RandomZoom(0.1),               # 第 4 章:縮放
    layers.RandomContrast(0.2),           # 第 5 章:對比
])

# --- 遷移學習:借用 ImageNet 預訓練的 MobileNetV2 ---
base = keras.applications.MobileNetV2(
    input_shape=(224, 224, 3),
    include_top=False,          # 去掉原本的分類層
    weights="imagenet")         # 載入預訓練權重
base.trainable = False          # 凍結:保留學好的通用特徵

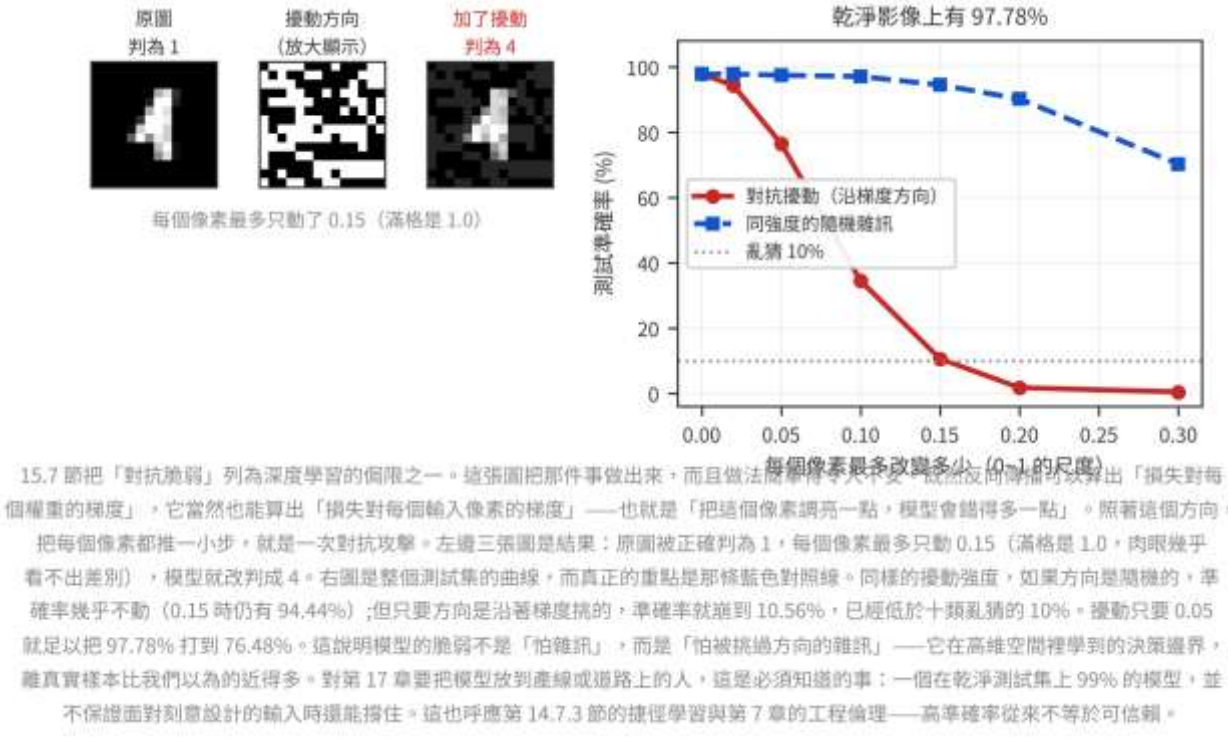
model = keras.Sequential([
    augment,                            # 先做資料增強
    base,                               # 借用的特徵萃取器
    layers.GlobalAveragePooling2D(),
    layers.Dropout(0.3),
    layers.Dense(5, activation="softmax")   # 換成自己的 5 類
])

model.compile(optimizer="adam",
              loss="sparse_categorical_crossentropy",
              metrics=["accuracy"])
# 只需少量自己的資料即可訓練出高效能模型
```

15.7 深度學習的侷限與展望

深度學習威力驚人，但絕非萬靈丹。理解它的侷限，是負責任地使用它的前提，也呼應了第 7、14 章一路強調的工程倫理。

第一，資料飢渴：深度學習需要大量標註資料，取得成本高昂;遷移學習與資料增強能緩解，但無法完全消除。第二，算力昂貴：訓練大型模型耗費可觀的電力與硬體，有其環境與成本代價。第三，黑箱難解：深度網路的決策過程難以解釋——它為什麼判斷這是貓?我們往往說不清。這在醫療、司法等高風險領域是嚴重問題(呼應第 14 章的肺炎 X 光捷徑陷阱：模型可能學到與答案相關卻無意義的捷徑，而我們難以察覺)。第四，對抗脆弱性：對影像加入人眼難以察覺的微小擾動，可能讓模型完全誤判——這在安全攸關的應用(如自駕、人臉辨識門禁)中是真實的風險。第五，分布偏移：模型在訓練分布外的資料上表現會急劇下滑(第 7.7.3 節已詳述)。



15.7 節把「對抗脆弱」列為深度學習的侷限之一。這張圖把那件事做出來，而且做法簡單得令人不安。我們沿著模型對輸入「損失對每個權重的梯度」，它當然也能算出「損失對每個輸入像素的梯度」——也就是「把這個像素調亮一點，模型會錯得多一點」。照著這個方向，把每個像素都推一小步，就是一次對抗攻擊。左邊三張圖是結果：原圖被正確判為 1，每個像素最多只動 0.15（滿格是 1.0，肉眼幾乎看不出差別），模型就改判成 4。右圖是整個測試集的曲線，而真正的重點是那條藍色對照線。同樣的擾動強度，如果方向是隨機的，準確率幾乎不動（0.15 時仍有 94.44%）;但只要方向是沿著梯度挑的，準確率就崩到 10.56%，已經低於十類亂猜的 10%。擾動只要 0.05 就足以把 97.78% 打到 76.48%。這說明模型的脆弱不是「怕雜訊」，而是「怕被挑過方向的雜訊」——它在高維空間裡學到的決策邊界，離真實樣本比我們以為的近得多。對第 17 章要把模型放到產線或道路上的人，這是必須知道的事：一個在乾淨測試集上 99% 的模型，並不保證面對刻意設計的輸入時還能撐住。這也呼應第 14.7.3 節的捷徑學習與第 7 章的工程倫理——高準確率從來不等於可信賴。

圖 15-20　對抗脆弱：看不出差別的影像，模型全認錯

展望未來，幾個方向正在改變格局：Transformer 架構(源於自然語言處理)已進入視覺領域(Vision Transformer)，在大資料上表現超越 CNN;自監督學習試圖擺脫對人

工標註的依賴，讓模型從無標籤資料中自學;而生成式 AI(擴散模型、多模態大模型)則模糊了「辨識」與「創造」的界線。但無論架構如何演進，本章的基礎——神經元、卷積、損失、梯度下降、反向傳播、以及對抗過度擬合的智慧——都是理解這一切的根基。技術的表層日新月異，底層的原理歷久彌新。

15.8 本章與全書的觀念連結總表

表 15-2 第 15 章與全書其他章節的觀念連結

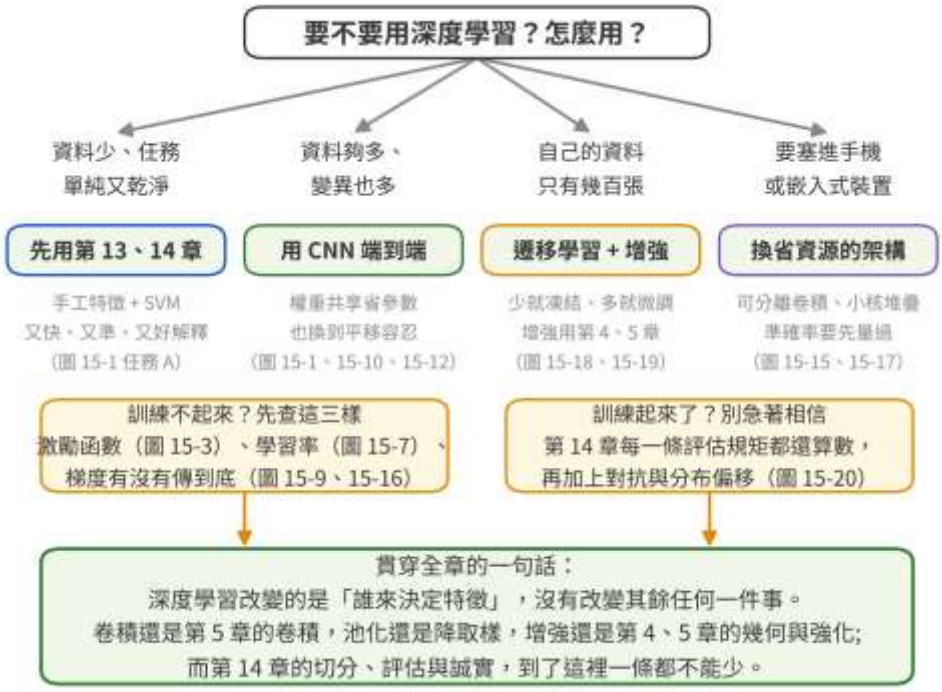


圖 15-21　第 15 章的決策地圖

程式實作 Lab 15

Lab 15-1 跨框架 MNIST 三部曲

目標：體會「架構思想共通、語法各異」。(a)以程式 15-1 用 Keras 訓練 MNIST CNN，達到 99% 以上準確率;(b)用 PyTorch 實作同樣架構的網路，比較兩者的程式風格(Keras 的宣告式 vs. PyTorch 的命令式);(c)用純 TensorFlow(低階 API)實作，體會框架自動化了多少細節;(d)三者訓練同一任務，比較準確率是否一致(應該相近)、程式碼行數與可讀性;(e)討論：為什麼說「不被單一工具綁架的核心理解力」比熟練某個框架

更重要?

Lab 15-2 卷積核視覺化

目標：親眼見證叫獸開講「網路自動學出特徵」的核心。(a)訓練一個 CNN 後，把第一層卷積核視覺化成小圖;(b)觀察它們是否自發演化成邊緣偵測器、方向性條紋(對照第 6 章 Gabor、第 13 章 SIFT);(c)把一張影像餵入網路，視覺化各層的「特徵圖」(feature map)，觀察淺層與深層分別對什麼有反應;(d)討論：這與第 13.8 節「CNN 重新發明手工特徵」的說法是否吻合?為什麼理解手工特徵有助於理解 CNN?

Lab 15-3 遷移學習實戰

目標：用少量資料訓練高效能模型。(a)收集自己的小資料集(如三到五類水果或零件，每類數十張);(b)以程式 15-2 借用 ImageNet 預訓練的 MobileNetV2，凍結特徵層、只訓練分類層;(c)對照組：用同樣的資料從頭訓練一個 CNN，比較兩者的準確率與訓練時間——遷移學習應在小資料上明顯勝出;(d)加入資料增強，觀察測試準確率的提升;(e)討論：對雲林在地的農業或製造業應用，為什麼遷移學習加資料增強常比從頭訓練更務實?

Lab 15-4 過度擬合的深度學習版

目標：在深度學習情境重現第 14 章的過度擬合診斷。(a)訓練一個 CNN，畫出訓練與驗證的準確率/損失隨 epoch 變化的曲線;(b)觀察過度擬合的典型徵兆：訓練準確率持續上升、但驗證準確率停滯甚至下降;(c)分別加入 Dropout、資料增強、及早停止，觀察各自如何改善過度擬合(呼應第 14 章四大武器);(d)討論：深度學習的過度擬合對抗手段，與第 14 章的傳統機器學習有何異同?為什麼深度網路特別容易過度擬合?

Lab 15-5 自己寫一次反向傳播

目標：把 15.2 節的公式變成你自己寫得出來的程式——這是理解深度學習最有效、也最一勞永逸的一堂課。全程只用 NumPy，不需要任何框架，也不需要 GPU。(a)實

作一個 Dense 層，提供 forward(x) 與 backward(dout) 兩個方法，分別對應 z = x·W + b 與 15.2.5 節那兩條式子;(b)實作 ReLU 與 Sigmoid 及其導數，再實作 Softmax 搭配交叉熵損失，確認最後一層的誤差訊號恰好是「預測機率減去 one-hot 答案」;(c)做梯度檢驗：把某個權重加減一個極小的 ε，用損失差除以 2ε 得到數值梯度，與反向傳播的結果比對相對差，確認中位數低於 1e−4（用 Tanh 做，理由見圖 15-8 與習題 17）;(d)用 sklearn 的 load_digits 訓練一個兩層網路，目標是測試準確率超過 95%;(e)把激勵函數全部換成 linear 重訓一次，並把各層權重相乘摺成單一矩陣，驗證它與原網路的輸出差異在浮點誤差等級（重現圖 15-4）;(f)把學習率設成 0.001、0.1、1.0、5.0 各訓練一次，畫出四條損失曲線，找出發散的那一個（重現圖 15-7）。延伸：再加上 Conv2D 與 MaxPool2D 兩層並通過梯度檢驗，你就擁有一個完整的迷你框架了。

Lab 15-6 小資料生存戰

目標：在只有幾百張影像的真實處境下，把能用的手段全部量過一遍，並學會用第 14 章的規矩誠實地報告結果。(a)建立基準線：收集或指定一個三到五類、每類數十張的小資料集（水果成熟度、零件良品與不良品皆可），先用第 13 章的 HOG 或顏色直方圖配 SVM 做出一個傳統基準，記錄準確率與訓練時間;(b)從頭訓練一個小 CNN，與基準比較——若 CNN 輸了，不要修飾結果，這正是圖 15-1 任務 A 的情境，請在報告裡解釋為什麼;(c)資料增強：加入旋轉、平移、亮度與對比調整（第 4、5 章），先確認每一種變換都不會改變標籤，再量出增益，並畫出「增益 vs 真實樣本數」的曲線（重現圖 15-19 的趨勢）;(d)遷移學習：借用一個預訓練模型，分別做「凍結特徵層」與「小學習率微調」兩種版本，在不同樣本數下比較三條曲線（從頭、凍結、微調），找出凍結開始變成天花板的交叉點（圖 15-18）;(e)誠實評估：以上每一個數字都要用交叉驗證跑至少三個隨機種子並附標準差;若兩個方法的誤差範圍重疊，就寫「分不出高下」，不要宣稱贏;(f)穩健性檢查：把測試影像平移幾個像素、改變亮度、或換一個拍攝場景再測一次，看看準確率掉多少，並說明這對上線部署的意義（圖 15-12、15-20）。延伸：挑你最好的模型做一次對抗測試——沿著損失對輸入的梯度方向加上肉

眼幾乎看不見的擾動，記錄準確率崩到多少，然後在報告最後寫三句話：這個模型可以上線嗎?為什麼?還缺什麼?

本章重點整理

- 深度學習的核心典範轉移：把特徵設計從人類手中交給機器，端到端一起學習;CNN 首層自發學出的濾波器與第 6、13 章的手工特徵(Gabor、SIFT)高度相似——深度學習是「自動重新發明」了手工特徵。
- 神經網路構件：神經元(加權求和加激活)、激勵函數(注入非線性，ReLU 為主流且緩解梯度消失)、損失函數(交叉熵/MSE 量化誤差)、梯度下降(往損失下坡走，學習率控制步長)、反向傳播(以連鎖律逆向算梯度，框架自動完成)。
- CNN 三大構件：卷積層(可學習的濾波核，局部連接、權重共享、層級特徵)、池化層(降取樣，即型態學膨脹/侵蝕)、全連接層(做最終決策);專為影像設計，解決 MLP 破壞空間結構與參數爆炸的問題。
- 架構譜系：LeNet(範式開創)→AlexNet(ReLU/Dropout/GPU，引爆革命)→VGG(3×3 加深)→GoogLeNet(Inception 多尺度)→ResNet(殘差連接，低於人類錯誤率)→MobileNet(深度可分離卷積，可邊緣部署);「愈深愈準」與「夠好可部署」兩條路線並存。
- 三大框架 TensorFlow(部署)、Keras(簡潔入門)、PyTorch(研究);跨框架 MNIST 實作培養不被工具綁架的核心理解;程式綜合了正規化、卷積池化、Dropout、驗證測試切分等全書觀念。
- 遷移學習借用預訓練模型的通用特徵、只重訓分類層，少量資料即可高效;資料增強(第 4、5、7 章的影像處理技術)對抗過度擬合。深度學習侷限：資料飢渴、算力昂貴、黑箱難解、對抗脆弱、分布偏移——負責任使用需正視這些。

習題

1. 深度學習相對於傳統影像辨識(第 13、14 章)，最革命性的改變是什麼?請以「特徵」說明。

   提示與參考解答：最革命性的是把「特徵設計」從人類手中交給機器：傳統流程由人設計特徵(SIFT、HOG)再餵給分類器，受限於人的設計能力;深度學習把「像素到特徵」與「特徵到類別」整合進同一網路端到端學習，自己學出最有助於任務的特徵。如叫獸開講，AlexNet 網路層自動長出邊緣偵測器與物件辨識器。

2. 為什麼激勵函數是必要的?若把所有激勵函數拿掉，深層網路會變成什麼?ReLU 相對 Sigmoid 有何優勢?

   提示與參考解答：激勵函數注入非線性。若拿掉，無論疊多少層都只是一連串線性運算的組合，等效於單一層，無法學複雜規律。ReLU(負值歸零、正值保留)相對 Sigmoid：計算極簡、且大幅緩解梯度消失問題，讓深層網路訓練又快又穩，是 AlexNet 的關鍵創新之一。

3. 用「地形與下坡」的比喻說明梯度下降。學習率太大或太小分別會怎樣?

   提示與參考解答：損失是一片高低起伏的地形，目標是走到最低點(最小損失)。梯度指出最陡上坡方向，往反方向(下坡)走一小步損失就下降，反覆走逼近谷底。學習率控制步長：太大會在谷底附近震盪甚至衝過頭，太小則收斂慢得令人絕望——又一個取捨。

4. 反向傳播解決了什麼問題?它的數學核心是什麼?為什麼理解它仍然重要(即使框架會自動完成)?

   提示與參考解答：它解決「如何算出每個權重對損失的影響(梯度)」的問題，使調整深層網路數百萬參數變得可行，是神經網路能訓練的數學引擎。核心是微積分的連鎖律：前向算損失，再從輸出往輸入逆向逐層傳遞誤差算梯度。框架雖自動完成，但理解它能幫助診斷訓練失敗的原因(如梯度消失或爆炸)。

5. 多層感知器直接處理影像有哪兩個致命問題?CNN 如何分別解決?

   提示與參考解答：問題一：把影像攤平破壞了空間結構(相鄰像素關聯)。問題二：參數量爆炸(小圖攤平就數萬輸入，全連接層權重上億)。CNN 解決：卷積的局部連接保留空間結構;權重共享(同一核滑遍全圖)大幅減少參數並帶來平移不變性。

6. CNN 的卷積層與第 5 章的空間濾波有何相同、有何關鍵不同?這說明了什麼?

提示與參考解答：相同：都是用小核在影像上滑動計算局部加權和，數學上是同一件事(卷積)。關鍵不同：第 5 章的核是人設計的(高斯、Sobel),CNN 的核是訓練學出來的——網路自己決定核裡的數值以萃取最有用的特徵。說明深度學習「自動學特徵」正是建立在傳統卷積之上，是它的自動化與升級。

7. 卷積層的「權重共享」帶來哪兩個好處?池化層有什麼作用?它與第 10 章的什麼運算相同?

提示與參考解答：權重共享(同一核滑遍全圖用同組權重)：其一大幅減少參數(一個 3×3 核只 9 個權重，不管影像多大)，其二帶來平移不變性(物件在哪都能被同一核偵測)。池化(如最大池化)降取樣：減少運算、擴大深層視野、提供位置容錯。最大/最小池化即第 10 章型態學的膨脹/侵蝕。

8. 請依序說明 CNN 架構譜系從 LeNet 到 MobileNet 各自的關鍵創新，並指出它們回應了什麼瓶頸。

提示與參考解答：LeNet(1998)開創卷積-池化-全連接範式。AlexNet(2012)以 ReLU/Dropout/GPU 引爆革命，回應「深層難訓練」。VGG(2014)全用 3×3 加深，證明深度價值。GoogLeNet(2014)Inception 多尺度加 1×1 壓縮，回應「更深卻要省參數」。ResNet(2015)殘差連接，回應「太深反而難訓練的退化問題」，錯誤率低於人類。MobileNet(2017)深度可分離卷積，回應「大模型無法在邊緣裝置執行」。

9. ResNet 的「殘差連接」解決了什麼問題?為什麼它讓訓練上百層的網路成為可能?

提示與參考解答：解決退化問題：網路太深時梯度難以傳遞、反而更難訓練、精度不升反降。殘差連接(skip connection)讓資訊可以「跳過」若干層直接往後傳，梯度也能沿捷徑順暢回傳，避免梯度消失，使訓練上百層網路成為可能。此思想至今無所不在(含 Transformer)。

10. 遷移學習的核心思想是什麼?為什麼它對「資料稀少」的應用特別有價值?

提示與參考解答：核心思想：預訓練模型較低的層已學會通用視覺特徵(邊緣、紋理、形狀)，對任何影像任務都有用、不必重學;因此保留(凍結)特徵萃取層，只重新訓練最後的分類層以適應自己的任務。對資料稀少的應用特別有價值，因為困難的「學特徵」已由 ImageNet 預訓練的巨人完成，只需少量自己的資料教它「特徵對

應到我的類別」。

11. 資料增強用到了本書前面哪幾章的技術?為什麼說「深度學習站在傳統影像處理的地基上」?

    提示與參考解答：資料增強用第 4 章幾何轉換(旋轉、平移、翻轉、縮放)、第 5 章空間域強化(亮度、對比)、第 7 章(加雜訊)。AlexNet 即以隨機裁切、翻轉、色彩擾動擴充資料。這說明深度學習不是憑空而來：資料增強是傳統影像處理、卷積源於空間濾波、正則化源於第 7 章、訓練評估原則源於第 14 章——深度學習穩穩站在前十四章的地基上，這正是本書把它放在最後的深意。

12. (綜合思辨題)深度學習有哪些主要侷限?為什麼在醫療、司法等高風險領域使用時要格外謹慎?請結合第 7、14 章的觀點。

    提示與參考解答：侷限：資料飢渴、算力昂貴、黑箱難解、對抗脆弱(微小擾動致誤判)、分布偏移(訓練分布外表現急降)。高風險領域謹慎的原因：黑箱使我們難以解釋模型為何判斷，可能學到與答案相關卻無意義的捷徑(第 14 章肺炎 X 光)、或在新分布上崩壞(第 7.7.3 節);清晰不等於正確(第 7 章還原 vs. 腦補)。工程師的責任不只調高準確率，更要理解模型學到什麼、資料是否公平——這是技術素養與工程倫理。

13. 有人說「深度學習一定比傳統影像處理好」。請用圖 15-1 的兩個任務反駁或限縮這句話，並說明深度學習真正的優勢出現在什麼條件下。

    提示與參考解答：這句話不成立，條件才是關鍵。圖 15-1 的任務 A 是乾淨、置中的 8×8 手寫數字，原始像素配 SVM 拿到 99.26%、HOG 配 SVM 95.56%，而端到端的 CNN 只有 98.33%——傳統管線還贏。任務 B 只改一件事：把同樣的數字擺到 14×14 畫布的隨機位置。原始像素 + SVM 立刻掉到 84.63%、HOG（已調到最佳參數）86.48%，而同一個 CNN（6,218 個參數）拿到 90.93%。差別來自權重共享：同一組卷積核滑遍整張影像，物件在哪裡都認得。所以深度學習的優勢出現在「任務夠難、變異夠多、資料夠多」時;在小而乾淨的問題上，第 13、14 章的工具往往又快、又準、又好解釋，而且不需要 GPU。工程上的判準是先看任務與資料，不是先看技術新不新。

14. 為什麼沒有激勵函數的深層網路等於單層?請說明數學理由，並設計一個實驗來驗證這件事。

提示與參考解答：線性映射的合成仍是線性映射——十個矩陣連乘仍然只是一個矩陣，所以十層全線性網路的表達能力與一層完全相同。驗證實驗：訓練一個十層全線性網路，再把各層權重依序相乘摺成單一矩陣 W 與偏置 b，比較原網路的輸出與 x·W + b。圖 15-4 實測兩者最大差異只有 1.78e−14，這個數量級就是浮點捨入誤差，等於說兩者是同一個函數。更有意思的是效能：一層線性拿到 96.11%，十層全線性（參數多 16 倍）只有 91.30%，不進反退，因為表達能力沒變、最佳化卻變難了。加回 ReLU 後，三層拿到 97.59%。結論：非線性不是裝飾品，它是深度之所以有意義的前提。

15. 梯度消失是怎麼產生的?請用連鎖律算出 Sigmoid 九層之後大約剩多少，並說明 ReLU 與殘差連接各自如何處理這個問題。

提示與參考解答：反向傳播每往回一層，誤差訊號就要乘上該層激勵函數的導數一次（15.2.5 節）。Sigmoid 的導數最大只有 0.25，所以九層之後上限約是 0.25 的九次方，大約四十萬分之一。圖 15-9 實測一個九層網路：最後一層梯度範數 5.09e−01，傳到第一層只剩 6.84e−06，相差約三萬七千倍，等於前面幾層幾乎收不到學習訊號;訓練四十輪後只有 56.48%，而 ReLU 版本 94.81%。ReLU 的處理方式是讓正半邊導數固定為 1，連乘不縮小。殘差連接則更徹底：區塊輸出是 x + F(x)，對 x 微分得到「1 加上 F 的導數」，那個 1 讓梯度永遠有一條直通路徑。圖 15-16 實測：無捷徑網路第一層梯度隨深度從 4.05e−01 縮到 8.73e−02，有殘差連接的反而從 6.98e−01 長到 1.27e+01。

16. 分類問題為什麼用交叉熵而不用均方誤差?請說明「錯得愈離譜、學得愈慢」這個現象的成因。

提示與參考解答：MSE 配 Sigmoid 輸出時，梯度要乘上 Sigmoid 的導數，而 Sigmoid 在兩端飽和、導數趨近 0，於是模型「錯得最離譜」的那些樣本反而梯度最小、學得最慢——這是最糟的組合。圖 15-6 量出這個差距：當正確類別的分數是 −6（錯得很離譜）時，交叉熵的梯度是 0.9976，MSE + Sigmoid 只有 0.002436，相差約 410 倍。後果是同一個網路訓練四十輪，交叉熵 96.67%、MSE 只有 50.00%。交叉熵搭配 Softmax 之所以成為標準配備，是因為兩者的導數恰好把飽和項約掉，最終的誤差訊號就是「預測機率減正確答案」，乾淨且不飽和。迴歸問題沒有這個飽和結構，MSE 仍然合理。

17. 什麼是梯度檢驗?請寫出它的做法，並說明為什麼在 ReLU 網路上會有少數參數「檢驗失敗」，那是不是程式有錯。

    提示與參考解答：梯度檢驗用數值微分驗證反向傳播的推導：把某個權重加上與減去一個極小的 ε，各算一次損失，相減再除以 2ε，得到與推導完全無關的數值梯度，再與解析梯度比對相對差，一般以 1e−4 為合格線。圖 15-8 對一個含卷積、池化與全連接層的網路抽查數百個參數：Tanh 版本中位數 2.61e−09、ReLU 版本 8.79e−10，都遠低於合格線。但 ReLU 版本有約 2.3% 的參數「失敗」，最差到 1.85e−01。這不是程式有錯，而是 ReLU 在 0 處有折點、不可微：當權重剛好讓 z 落在 0 附近，加減 ε 會跨過折點，數值梯度量到的是折點兩側的平均斜率，本來就與解析梯度不同。Tanh 處處平滑，完全沒有這個現象。實務規矩：梯度檢驗用平滑激勵函數做，並看中位數與絕大多數參數，而不是單一最差值。

18. CNN 的權重共享帶來「平移不變性」——這個常見說法需要修正成什麼?請以圖 15-12 的實測說明，並說明真正的不變性從哪裡來。

    提示與參考解答：嚴格說，卷積給的是「平移等變」（影像平移，特徵圖跟著平移），加上池化之後換到一點「平移容忍」，但不是不變。圖 15-12 實測（三個模型都只用置中的數字訓練）：平移 2 個像素時，MLP 崩到 13.52%（十類亂猜是 10%）、CNN 還有 44.63%，CNN 確實好得多;但平移 4 個像素時 CNN 也只剩 12.41%，一樣崩了。真正的不變性來自資料：同一個 CNN 若訓練時就看過隨機平移的樣本（資料增強，第 4 章幾何轉換），平移 3 個像素仍有 89.07%，是未增強版本 25.93% 的 3.4 倍。代價是置中時略低（94.63% 對 97.78%），因為同樣的容量要分給更多情況。結論：不變性多半是資料教出來的，不是架構免費送的。

19. 池化層付出什麼代價、換到什麼好處?請用圖 15-13 的四個數字說明這筆交易划不划算。

    提示與參考解答：代價是準確率從 98.33% 降到 97.96%，少了 0.37 個百分點。換到的有三樣：其一，展平後的維度從 3,136 降到 144，總參數從 101,962 降到 6,218，少了約 16 倍;其二，預測時間快約 1.6 倍;其三，平移 2 個像素時的準確率從 11.1% 升到 44.1%，約 4 倍的平移容忍。此外池化還擴大感受野：3×3 卷積感受野是 3，加一次 2×2 池化後，下一個 3×3 卷積看到的相當於原圖 10 個像素，再池化一次是 20 個——這是深層能「看見整體」的關鍵（15.3.4 節）。用 0.37 個百分點換這些，在幾乎任何實務場合都划算，尤其是第 17 章要把模型塞進邊緣裝置時。

20. 為什麼 VGG 要把所有卷積核都改成 3×3?請算出兩個 3×3 與一個 5×5 的感受野與參數量，並說明小核堆疊除了省參數還多賺了什麼。

    提示與參考解答：兩個 3×3 串接的感受野是 1 + 2×(3−1) = 5，與一個 5×5 相同;三個 3×3 則等於一個 7×7。參數量方面（以 256 通道的層計算）：一個 5×5 要 1,638,400 個，兩個 3×3 只要 1,179,648 個，省 28%;一個 7×7 要 3,211,264 個，三個 3×3 只要 1,769,472 個，省 45%（圖 15-15）。多賺的是非線性次數：一個 5×5 後面只接一次激勵函數，兩個 3×3 之間夾一次、後面還有一次，共兩次，所以表達能力更強。通式是：堆 n 個 3×3 的參數量是單一等效大核的 $9n/(2n+1)^2$，n 愈大省愈多。這是少見的「省成本又提升能力」，也是 VGG 之後幾乎所有架構都改用小核的原因。

21. ResNet 要解決的「退化問題」是什麼?它與過度擬合有何根本不同?請用圖 15-16 的訓練準確率說明如何分辨這兩者。

    提示與參考解答：退化問題是指網路加深到一定程度後，精度不升反降。它與過度擬合的根本差別在訓練表現：過度擬合是「訓練好、測試差」;退化是「訓練也差」，表示模型根本沒學起來，是最佳化困難而非泛化問題。圖 15-16 實測：26 層的無捷徑網路訓練準確率只有 61.34%、測試 60.19%，兩者都差——這是退化不是過擬合;同樣 26 層加上殘差連接，訓練 97.61%、測試 93.70%。分辨方法因此很簡單：先看訓練準確率。訓練也上不去，就別往正則化或資料增強的方向調，該檢查的是梯度傳不傳得下去、學習率、初始化與架構（15.2.5 節）。

22. 遷移學習什麼時候該凍結特徵層、什麼時候該解凍?請用圖 15-18 的實測說明，並用偏差-變異的語言解釋為什麼會有交叉點。

    提示與參考解答：資料少時凍結、資料多時解凍。圖 15-18 實測：每類 5 張（共 25 張）時，凍結式遷移 38.17%、從頭訓練 32.29%，遷移領先 5.9 個百分點;但每類 40 張（共 200 張）時，凍結式 68.97% 反而輸給從頭訓練的 74.85%，落後 5.9 個百分點。用偏差-變異解釋（第 14.6.4 節）：凍結大幅減少可訓練參數，因而壓低變異，但同時把偏差鎖住——被凍結的特徵學不到目標任務特有的東西。資料少時「少一點變異」值得用「多一點偏差」去換;資料多到足以自己學出更貼合的特徵時，凍結就變成天花板。實務上的標準答案是微調：不凍結、但用較小的學習率，實測在小資料時接近凍結式、資料變多時追上從頭訓練，兩邊都不吃虧。交叉點在哪裡只能靠驗證集找。

23. 資料增強為什麼有效?它的增益會隨什麼改變?請說明選擇增強方式時最重要的前提。

> *提示與參考解答：深度模型參數多、偏差低但變異高，而增強利用「小幅旋轉、平移、亮度變化不改變標籤」這個領域知識，把原本要靠資料量才學得到的不變性直接寫進訓練集，藉此壓低變異（15.6.3 節）。圖 15-19 實測：只有 50 張真實樣本時，增強到四倍讓準確率從 32.73% 升到 46.04%，多 13.3 個百分點;100 張時增益 17.4 個百分點;到 400 張時縮到 8.4 個百分點。增益隨真實資料量增加而遞減，因為增強出的樣本彼此高度相關，提供的新資訊遠少於等量真實資料。最重要的前提是「標籤不變」：對手寫數字做上下翻轉會把 6 變成 9，對醫學影像左右翻轉可能把心臟換到右邊——那不是增強，是加錯誤標籤。選擇增強方式靠的是領域知識，不是演算法。*

24. (推導題)什麼是對抗樣本?它為什麼能用反向傳播做出來?請說明「模型怕的不是雜訊，而是被挑過方向的雜訊」這句話的實測根據。

> *提示與參考解答：對抗樣本是加了微小、刻意設計的擾動後能讓模型誤判的輸入。既然反向傳播能算出損失對每個權重的梯度，它同樣能算出損失對每個輸入像素的梯度，也就是「這個像素往哪個方向改，模型會錯得更多」;沿著這個方向把每個像素推一小步，就是一次攻擊。圖 15-20 的實測給出那句話的根據：在相同的擾動強度下，若方向是隨機的，準確率幾乎不動（每像素最多改 0.15 時仍有 94.44%）;若方向沿著梯度挑選，準確率崩到 10.56%，已低於十類亂猜。擾動只要 0.05 就足以把 97.78% 打到 76.48%。這說明脆弱性是「方向性」的：模型在高維空間學到的決策邊界，離真實樣本比我們以為的近得多。對第 17 章要把模型放上產線或道路的人，這表示乾淨測試集上的高準確率不等於可信賴，呼應第 14.7.3 節的捷徑學習與第 7 章的工程倫理。*

25. (綜合實作思辨題)你要為一家中小型零件廠做表面瑕疵檢測，手上有 300 張已標註影像、一台沒有獨立顯卡的工控電腦，客戶要求「漏檢率愈低愈好」。請規劃完整方案，說明每個決定的依據，並指出三個最可能失敗的地方。

> *提示與參考解答：方案應逐項說明依據。(1)技術選擇：300 張屬於小資料，且工控電腦沒有 GPU，因此優先考慮「第 13 章特徵 + SVM」與「遷移學習 + 微調」兩條路線並實測比較，而不是從頭訓練大型 CNN（圖 15-1、15-18）。(2)小資料對策：資料增強必用，且只用保持標籤不變的變換（旋轉、平移、亮度;若瑕疵有方向*

性則不可翻轉）——圖 15-19 顯示小資料時可多出十幾個百分點。遷移學習先凍結、資料累積後改微調。(3)部署考量：無 GPU 代表要控制模型大小與延遲，可評估深度可分離卷積與小核堆疊，但必須實測準確率代價（圖 15-17 顯示小網路上換成可分離版本反而掉了 5.7 個百分點），第 17 章詳述。(4)評估：客戶要求「漏檢率愈低愈好」等於要高召回率，因此用混淆矩陣、精確率與召回率，並依代價調整判定門檻，不能只看準確率（第 14.5.2 節、圖 14-13、14-15）;資料量小務必用交叉驗證並報標準差（第 14.5.3 節）。三個最可能失敗的地方：其一，資料偏差與捷徑——300 張若來自同一批次、同一光源，模型可能學到拍攝條件而非瑕疵（第 14.7.3 節、圖 14-20、15-20 的分布偏移），對策是跨批次、跨光源收集驗收資料;其二，類別不平衡——瑕疵品通常遠少於良品，準確率會被灌水（圖 14-13），對策是看召回率、設類別權重、調門檻;其三，樣本數不足以支撐可信的結論——測試集若只有數十張，標準誤可達數個百分點，任何小幅改善都不可信（第 14.5.3 節）。最後應建立上線後的持續抽檢與回饋機制，因為製程與光源會隨時間漂移。

參考資料與延伸閱讀

- Goodfellow, I., Bengio, Y., & Courville, A., Deep Learning, MIT Press, 2016.(深度學習權威教科書)
- Krizhevsky, A., Sutskever, I., & Hinton, G. E., ImageNet Classification with Deep Convolutional Neural Networks, NeurIPS, 2012.(AlexNet，本章叫獸開講之史料來源)
- LeCun, Y., Bengio, Y., & Hinton, G., Deep Learning, Nature, 2015.
- Simonyan, K., & Zisserman, A., Very Deep Convolutional Networks (VGG), ICLR, 2015;He, K., et al., Deep Residual Learning (ResNet), CVPR, 2016.
- Howard, A. G., et al., MobileNets: Efficient CNNs for Mobile Vision Applications, arXiv, 2017.
- OpenCV、TensorFlow、Keras、PyTorch 官方文件與教學。
- 補充教材投影片下載：https://page.line.me/prof(機器人叫獸)